AOSP 进阶丛书 · 第五册 · ANDROID 13 (API 33)

WMS 窗口与输入体系

窗口层级树与 addWindow 审验漏斗、performTraversals 三幕剧、Choreographer 双 VSync 与三缓冲、触摸事件从 evdev 到 onTouchEvent 的七段全链路——像素与指尖各得其所。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：让像素与指尖各得其所 2

第一章 窗口层级：WMS 眼中的屏幕 2

第二章 窗口的诞生与画布 4

第三章 一帧的编排：performTraversals 三部曲 6

第四章 Choreographer 与 VSync：时间的实体化 8

第五章 输入全链路：从指尖到 onTouchEvent 9

第六章 焦点、触摸目标与多屏 13

第七章 实战：一次掉帧排查全记录 14

第八章 深水区：交互系统的四条支线 14

第九章 渲染衔接：RenderThread 与 SurfaceFlinger 16

第十章 设计决策八问 18

附录 A 观测实操 19

附录 B 自检题 20

附录 C 快问快答（面试扫雷 20 条） 20

第十一章 同步屏障深潜：一帧的优先级引擎 21

第十二章 WindowState 的一生：出生、装户与死亡清理 27

第十三章 输入窗口的户籍簿：InputMonitor 与焦点裁决 33

第十四章 转屏全链路：冻结、旋转与解冻 37

教科书习题集（分级） 43

术语表（按拼音排序） 45

排障手册：WMS 窗口与输入体系问题定位 47

附录 E：窗口/输入/绘制核心机制深度速查表 49

本册速览

窗口层级树与 addWindow 审验漏斗 · z 序三定律
performTraversals 三幕剧 · 同步屏障 · Choreographer 四车道
触摸七段全链路（evdev→onTouchEvent） · 5s ANR 回环
深水区：按键拦截链/预测返回/传感器 FMQ/IME/无障碍
渲染衔接：RenderThread/SF/SurfaceView/动画接入

前言：让像素与指尖各得其所

第四册结束时，Activity 站上了舞台但还没开演。本册接棒"首帧之后"的世界：**窗口如何被系统登记与分层（WMS）、一帧如何被编排（Choreographer/performTraversals）、指尖事件如何穿越内核抵达你的 onTouchEvent（inputflinger）**。这三件事共享同一条时间轴——VSync，本册会把它讲成一个可以掐表验证的实体。

体例沿用第四册：源码逐段注释 + 逐段精读 + 全链路图。锚点基于本树 Android 13 实测；inputflinger（native）与 View 体系（Java）的双语世界是本册特色——你将习惯在两种语言间跳栈。

第一章 窗口层级：WMS 眼中的屏幕

设计洞察 · 学习目标

1. 用两棵树（任务树/窗口树）与 WindowToken 解释窗口准入；2. 背出 z 序三定律并说出 type 层基线区间；3. 走读 addWindow 四步审验漏斗并对应 BadToken/PermissionDenied 两类崩溃；4. 解释 Insets 协商模型与 adjustResize/adjustPan 的差异

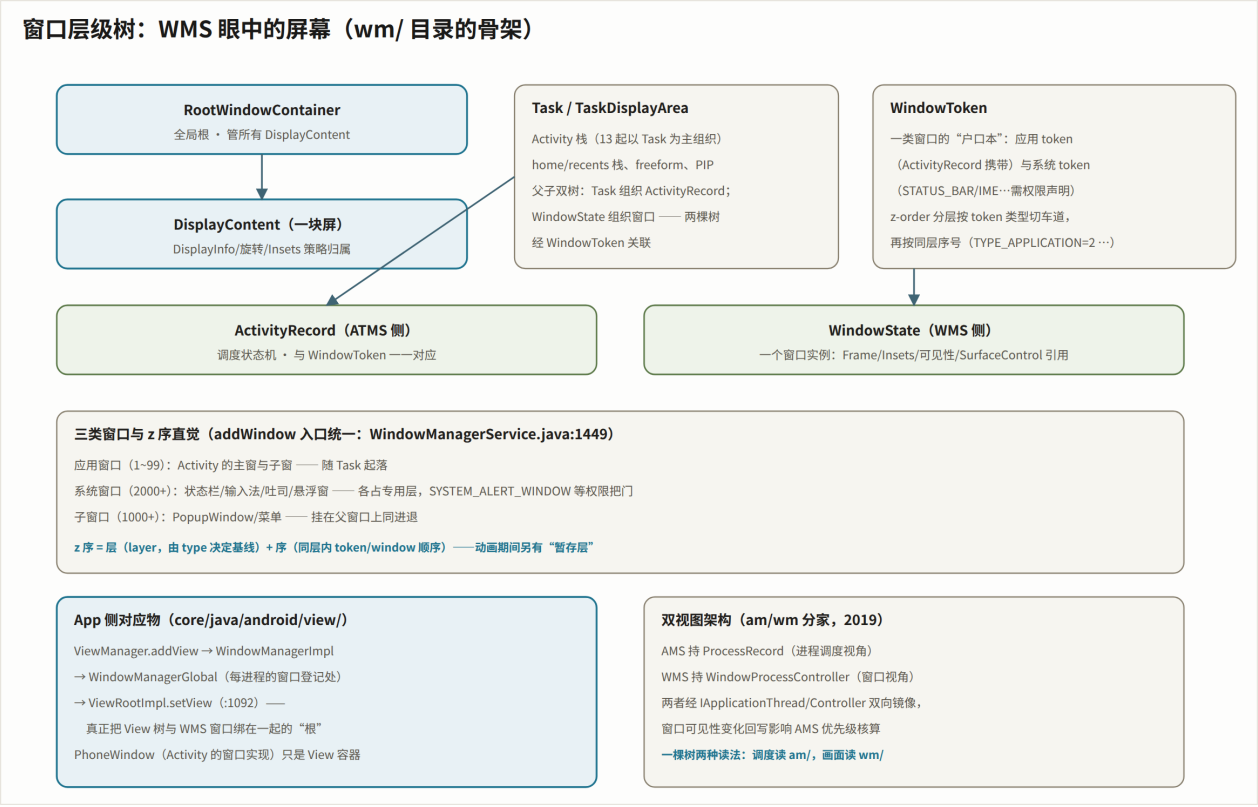


图 1 窗口层级树：RootWindowContainer→DisplayContent→Task/WindowToken→WindowState，及双视图架构

1.1 两棵树一个户口本

WMS 用**两棵树**管理屏幕：任务树（Task 组织 ActivityRecord，决定"哪个任务在前"）与窗口树（WindowState 组织真实窗口，决定"谁盖着谁"）。`WindowToken` 是两棵树的缝合点——Activity 启动时（第四册 §1.3）预先注册应用 token，`addWindow` 时窗口必须持有效 token 才准入。系统类窗口（状态栏/输入法）各有专用 token 与独立层位，这是"悬浮窗为什么需要权限"的机制根源：**token 即户口，户口即层位**。

1.2 z 序三定律

z 序 = `layer`（由窗口 type 决定基线：应用 1~99 / 子窗 1000+ / 系统 2000+）+ 同层内 token 与窗口顺序 + 动画期间的暂存层。三句话记住调度行为：**type 定车道**（SYSTEM_ALERT 永远在应用之上）；**Task 序定同车道名次**（前台任务的窗口整体靠上）；**动画短暂例外**（窗口动画期间层位由动画器接管，结束归还）。

> **源码锚点 · 第一章锚点**
> WindowManagerService.java:1449 addWindow / :2219 relayoutWindow
ViewRootImpl.java:1092 setView（App 侧根）
窗口类型区间：core/java/android/view/WindowManager.LayoutParams（TYPE_* 注释即层级说明书）

1.3 WMS 侧 addWindow 逐段注释（真实骨架）

java frameworks/.../wm/WindowManagerService.java:1449 起（骨架+讲解注释）

```
public int addWindow(Session session, IWindow client, LayoutParams attrs, int viewVisibility,
int displayId, ...) {
// 讲解注释①：定位 DisplayContent（多屏时决定"这块屏收不收这个窗口"）。
// 讲解注释②：token 审验：attrs.token 必须能对上已注册的 WindowToken
//  （应用的 token 由 Activity 启动时预建；对话框等子窗口须挂父窗口 token）。
// 对不上 → ADD_BAD_APP_TOKEN/ADD_NOT_APP_TOKEN 等错误码回流 App。
// 讲解注释③：类型与权限：系统窗口逐类型查权限
//  （SYSTEM_ALERT_WINDOW/STATUS_BAR_SERVICE...），缺权限 → ADD_PERMISSION_DENIED。
// 讲解注释④：new WindowState（一切窗口事实的载体：Frame/Insets/可见性/
// SurfaceControl 占位），attach 到 WindowToken 与 DisplayContent 树。
// 讲解注释⑤：输入通道配对：openInputChannel → server 端注册给
// InputDispatcher，client 端随返回值交给 App（第五章起点）。
// 讲解注释⑥：焦点/层序重排：assignWindowLayers，必要时触发
// InputMonitor 焦点更新（第六章 key 寻址变化）。
return ADD_OKAY;
}
```

逐段精读：②③是开发者两大类崩溃的判决现场（BadToken/PermissionDenied）；⑤把"窗口存在"与"可收输入"原子化绑定——**没有输入地址的窗口即使可见也收不到事件**（触摸仍可命中但走不了通道的场景另有 overlay 处理）。四步审验（屏→token→权限→建籍）的顺序本身就是安全语义从粗到细的漏斗。

1.4 Insets：内容与遮挡物的谈判

状态栏、导航栏、输入法对内容区的"侵占"不是硬裁剪，而是一场持续协商：`InsetsController`（App 侧）与 WMS 的 `InsetsState` 按 **InsetsType 分通道**谈（statusBars/navigationBars/ime 各自独立），窗口可用 `behavior`（DEFAULTS/TRANSPARENT/none）声明姿态。键盘弹出动画即一次 Insets 动画协商：系统侧与 App 侧**同帧同步**跑（`InsetsAnimationThread` 的存在让动画不被主线程卡顿拖累）。

逐段精读一个高频现象："布局被键盘顶起 vs 平移躲开"——`windowSoftInputMode=adjustResize` 是 Insets 全量应用于内容（重新 traversal）；`adjustPan` 是窗口整体平移（不重测）。两种模式对应两种协商结果，都不是"键盘画在你的窗口上"——**键盘是独立窗口，只是协商出的裁剪量进入你的测量参数**。

第二章 窗口的诞生与画布

设计洞察 · 学习目标

1. 走读 setView 五步：落户、预演、通道、跨进程登记、挂根；2. 解释 relayout 如何交付 SurfaceControl 与"职责三分离"；3. 说清 Window/PhoneWindow/DecorView/ViewRootImpl 四层关系；4. 用"解析、演算、呈现三阶段分离"解释 onCreate 里 getWidth()==0 等时序困惑



图 2 addWindow 与 relayout 双时序：户口登记、SurfaceControl 交付与职责三分离

2.1 setView 逐段注释（App 侧第一站）

java frameworks/base/core/java/android/view/ViewRootImpl.java:1092-1180（骨架+讲解注释）

```
public void setView(View view, WindowManager.LayoutParams attrs, View panelParentView,
int visibility) {
synchronized (this) {
if (mView == null) {
mView = view;
// 讲解注释①：布局参数落户：attrs.type 决定窗口类型，
// 此后 WindowManagerGlobal（进程级登记处）把本 ViewRootImpl 纳管。
...
// 讲解注释②：首次 measure/layout 在"无窗口"状态下预演——
// 让 WMS 拿到想要的尺寸（View 还没上屏）。
requestLayout();
...
// 讲解注释③：输入通道建立：mInputChannel 随 addToDisplay 返回，
// 从此指尖事件有了邮寄地址（第五章全链路的 App 端起点）。
if ((mWindowAttributes.inputFeatures
& INPUT_FEATURE_NO_INPUT_CHANNEL) == 0) {
mInputChannel = new InputChannel();
}
...
// 讲解注释④：res = mWindowSession.addToDisplay(...)：
// 跨进程登记（Session 每 App 一条），返回 ADD_OKAY 或错误码
//  （BAD_TOKEN_... 类错误在此暴露给开发者）。
...
// 讲解注释⑤：View 树挂接：view.assignParent(this)——
// 此后 invalidate 的"冒泡"终点就是本类（第三章的入口）。
view.assignParent(this);
}
}
}
```

逐段精读：②与④解释了开发者常见困惑——onCreate 里 getWidth()==0（还没演算）与 BadTokenException（token 未就绪，典型如对话框在 Activity attach 前弹出）。⑤是理解事件流的钥匙：**ViewRootImpl 既是 View 树的"根"，也是整个窗口的"总管"**。

2.2 relayout：拿画布

performTraversals 需要画布时走 Session.relayout → WMS.relayoutWindow（:2219）：按请求尺寸/可见性更新窗口，**创建 SurfaceControl**（SurfaceFlinger 世界的图层句柄）跨进程交付。职责三分离从此定形：**WMS 管户口与层位，App 管内容生产（往 Surface 画），SF 管合成上屏**——系统从不碰你的像素，只调度你的画布。

2.3 PhoneWindow 与 DecorView：Activity 的"内脏"

Activity 并不直接持有 ViewRootImpl——中间还有两层：**PhoneWindow**（core/java/com/android/internal/policy/PhoneWindow.java，Window 的唯一实体实现）管理 DecorView 与标题/内容区模板；**DecorView** 是整棵 View 树的根容器（setContentView 实为 installDecor→generateLayout：按主题特性选装饰模板再挂你的 layout）。

逐段精读一条冷知识：`findViewById` 为什么 onCreate 里就能用？——`setContentView` 已完成 LayoutInflater 解析整棵树（**解析不等同于测量**，尺寸仍是 0，第三章解释）；而"看见了"要等首次 traversal + relayout 提交首帧。**解析（结构）、演算（尺寸）、呈现（像素）三阶段分离**，是理解所有"UI 时序问题"的分界线。

LayoutInflater 深一层：`inflate` 走 `XmlPullParser`+反射构造（`tryCreateView` 优先 Factory2 钩子——AppCompat 借此偷换控件实现），`merge`/`ViewStub` 是解析期优化原语。面试常考 `inflate(layoutRes, root, attachToRoot)` 三参语义：root 决定"要不要 layout_params 与挂接"，attachToRoot 决定"立即挂还是交给调用方"。

第三章 一帧的编排：performTraversals 三部曲

设计洞察 · 学习目标

1. 用欠条模型解释 scheduleTraversals 的三步：去重、屏障、排队；2. 说出三幕剧的规矩与各幕禁忌；3. 走读 performTraversals 决策段的关键判断；4. 解释 wrap_content/weight 引发两次测量的机制



图 3 Choreographer 回调队列与 performTraversals 三幕剧全景

3.1 欠条模型：scheduleTraversals（真实源码逐段注释）

java ViewRootImpl.java:2257-2286（节选+讲解注释）

```
void scheduleTraversals() {
if (!mTraversalScheduled) { // 讲解注释①：同一帧内只欠一次账
```

```
mTraversalScheduled = true; // （多次 invalidate 合并为一帧）
mTraversalBarrier = mHandler.getLooper().getQueue().postSyncBarrier();
// 讲解注释②：同步屏障——插队挡住普通消息，
// 让 vsync 消息（异步）优先被处理
mChoreographer.postCallback(
Choreographer.CALLBACK_TRAVERSAL, mTraversalRunnable, null);
// 讲解注释③：排队到遍历车道（:2262），
// 等 VSync 唤醒 doTraversal(:2286)
...
}
}
```

逐段精读：②的**同步屏障**是面试高频——postSyncBarrier 让 Looper 队列跳过普通消息直取异步消息，确保 vsync 到达后 traversal 立刻执行，不被积压的普通消息拖延。拿掉屏障（removeSyncBarrier）发生在 doTraversal 里。**“插队合法化"是帧稳定性的基础设施。**

3.2 三幕剧的规矩

第一幕 measure（performMeasure:2541）：自顶向下发放 MeasureSpec（父约束+子期望），子 View 在 onMeasure 里"申报"尺寸；wrap_content 常引发二次 measure。第二幕 layout：自顶向下"拍板"位置，此阶段再 requestLayout 直接抛异常（防递归震荡）。第三幕 draw：软件画布逐条立即执行；硬件加速先录 DisplayList（RecordingCanvas），RenderThread（libhwui）异步提交 GPU——**主线程负责"画什么"，渲染线程负责"怎么画"**。

先修要求与学习路线

先修：已读前三册（架构/启动/Binder）。

读完本书你将能够：

- 追踪WMS 窗口与输入体系的核心机制的完整调用链
- 解释关键设计决策的"为什么"
- 使用 dumpsys 定位本领域问题
- 回答相关面试题（书7 对应章节）

> **设计洞察 · 初学者最易误解的三件事**
> ① invalidate() 不立刻重绘，只是欠条（等下一个 VSync）；② requestLayout() 不触发 draw，draw 由合并判断决定；③ UI 线程"单线程"约束来自 ViewRootImpl.checkThread——单线程免锁的代价，而非性能优选。

3.3 performTraversals 决策段逐段注释（真实骨架）

java ViewRootImpl.java:2286 起（骨架+讲解注释）

```
private void performTraversals() {
// 讲解注释①：期望尺寸演算：mWinFrame（WMS 拍板的上次帧）与
// mWindowAttributes 尺寸愿望取交集；首次布局用 dipsToPixels 推
```

```
// 首选宽高（所以第一帧前 getWidth()==0）。
// 讲解注释②：是否需要 relayout？尺寸/insets/可见性任一变化
// → relayoutWindow（第二章：拿新 SurfaceControl）。
// 讲解注释③：三幕剧开演（measure→layout→draw），脏区 mDirty 并集优化。
// 讲解注释④：收尾：mFirst 已清；若执行中被再次 invalidate，
// scheduleTraversals 已记新账——下一 vsync 续演。
}
```

设计洞察 · 逐段精读：MeasureSpec 两次 measure 之谜

父容器对 wrap_content 子 View 的测量分两轮：第一轮按 UNSPECIFIED 探"你想要多大"，第二轮以 AT_MOST 限定"最大这么大"。LinearLayout 的 weight、RelativeLayout 的依赖都是多轮测量的推手——**自定义 View 必须保证 onMeasure 幂等且不留副作用**，否则偶发布局漂移几乎无法排查。

第四章 Choreographer 与 VSync：时间的实体化

设计洞察 · 学习目标

1. 走读 doFrame 对表与四车道顺序（INPUT→ANIMATION→INSETS→TRAVERSAL）；2. 用三缓冲模型解释掉帧为何不必然可见；3. 读懂 FrameTimeline 表并指认掉帧责任方；4. 区分主线程与 RenderThread 两个时钟域（getSfInstance）

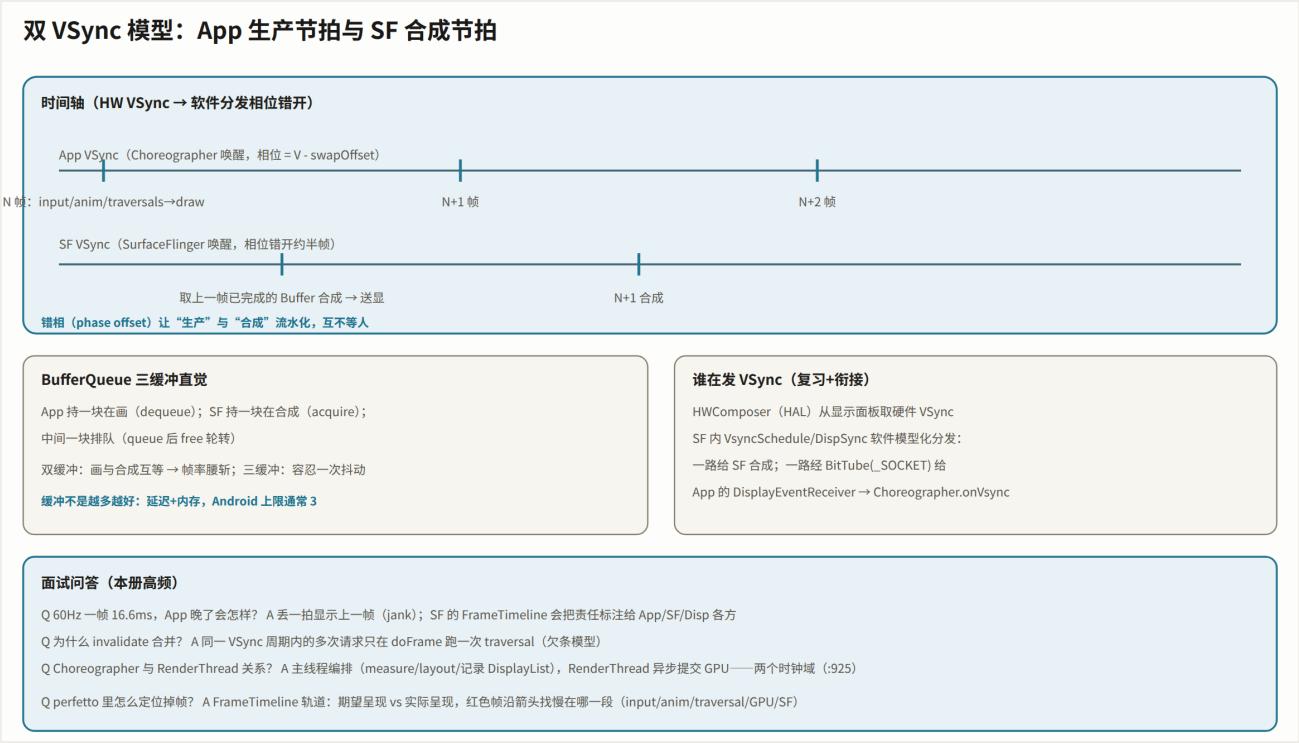


图 4 双 VSync 相位错开模型、BufferQueue 三缓冲与掉帧责任判定

4.1 doFrame 逐段注释

java core/java/android/view/Choreographer.java doFrame（骨架+讲解注释）

```
private void doFrame(long frameTimeNanos, int frame,
DisplayEventReceiver.VsyncEventData vsyncEventData) {
// 讲解注释①：对表：frameTimeNanos 是 vsync 的"官方时刻"，
// 若本帧来晚了（doFrame 调度被延迟），先算 skipped 次数补偿跳过的帧。
...
// 讲解注释②：按车道顺序发车：INPUT(0) → ANIMATION → INSETS → TRAVERSAL(3)
// （常量定义 :226/:256）——同帧内"先处理输入，再算动画，最后测量布局"，
// 保证这一帧的画面已经消化完这一帧的输入。
doCallbacks(Choreographer.CALLBACK_INPUT, frameTimeNanos);
doCallbacks(Choreographer.CALLBACK_ANIMATION, frameTimeNanos);
...
doCallbacks(Choreographer.CALLBACK_TRAVERSAL, frameTimeNanos);
}
```

逐段精读：车道顺序是**体感流畅的因果链**——先输入后画面，同帧闭环。若顺序颠倒（先画后输入），触控延迟会恒定多一帧。`ViewRootImpl` 持 UI 线程实例；`getSfInstance`（:925）是 RenderThread 专用的第二时钟域——**主线程编排、渲染线程执行的"双时钟"是硬件加速时代的标配**。

4.2 三缓冲直觉

App 持一块在画（dequeue）、SF 持一块在合成（acquire）、一块在排队（queued）。双缓冲会互相等待（帧率腰斩），三缓冲多花一份内存买回"容忍一次抖动"。`dumpsys SurfaceFlinger --latency`（或 perfetto 的 FrameTimeline）能直接看到这个轮转——掉帧责任（App 慢/SF 慢/显示晚）沿时间线逐帧标注。

4.3 FrameTimeline：掉帧判读实例

text perfetto FrameTimeline（示意，字段真实）

```
Frame# expectedPresent actualPresent jank 责任
4812 10:00:01.000 10:00:01.000 None —
4813 10:00:01.016 10:00:01.033 Jank(App) ← traversal 22ms（App 慢）
4814 10:00:01.033 10:00:01.033 None ← 追帧成功（三缓冲吸收）
4815 10:00:01.050 10:00:01.083 Jank(SF) ← SF 合成 20ms（SF 慢）
```

读表三步：先对齐 expected 与 actual（差一格=晚一帧）；再看 jank 标签的责任方（App/RenderThread/SF/Disp）；最后下钻对应轨道（traversal 分段、GPU completion、SF 合成队列）。**"卡顿是谁的锅"在这张表里是可指认的事实而非直觉**——这也是本册建议每位读者亲手抓一次 trace 的原因。

第五章 输入全链路：从指尖到 onTouchEvent

设计洞察 · 学习目标

1. 说出输入三线程（EventHub/InputReader/InputDispatcher）的分工与隔离价值；2. 走读七段链路并标注每次跨进程与 ANR 计时点；3. 解释 InputStage 责任链与 IME 截胡；4. 用 mFirstTouchTarget 四规则推导滑动冲突方案

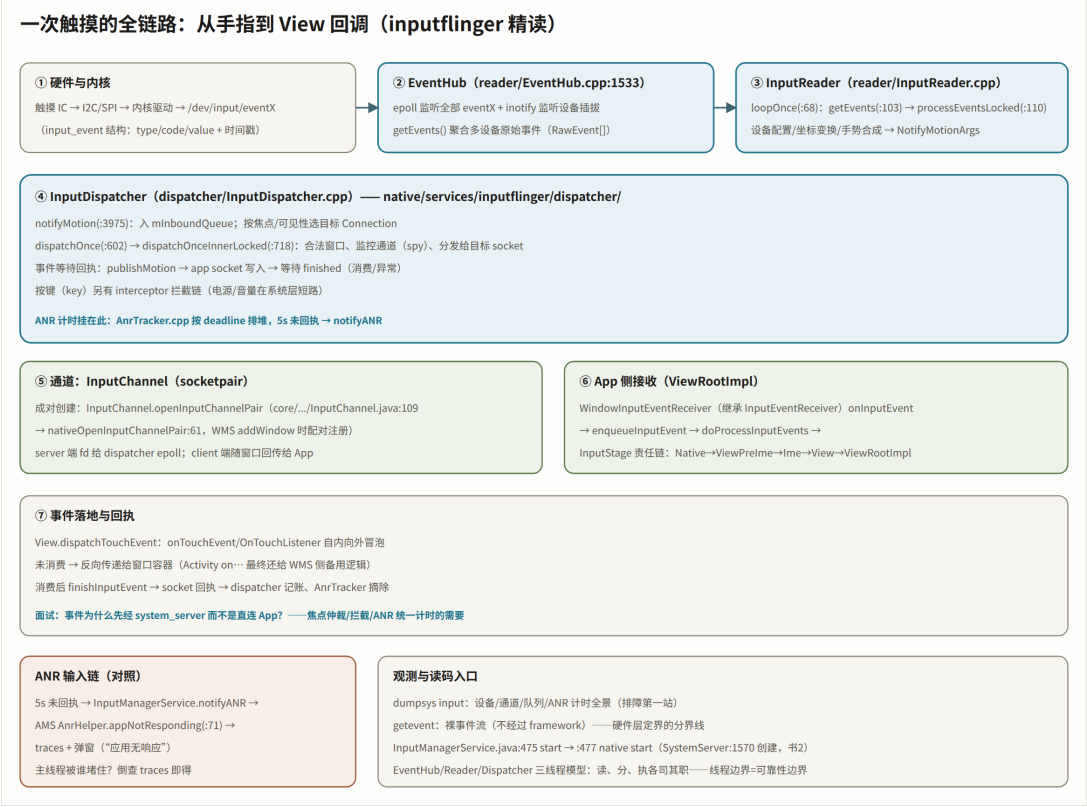


图 5 触摸事件全链路：内核 evdev→EventHub→InputReader→InputDispatcher→InputChannel→ViewRootImpl→View，含 5s ANR 回环

5.1 三线程分工（native/services/inputflinger/）

EventHub（`EventHub.cpp:1533` getEvents）epoll 全部 `/dev/input/eventX`，inotify 管设备插拔——原始事件批发商。**InputReader**（`InputReader.cpp:68` loopOnce → `:103` getEvents → `:110` processEventsLocked）做设备配置、坐标变换、手势合成（移动/点击合成、多点归并），产出 `NotifyMotionArgs`。**InputDispatcher**（`InputDispatcher.cpp:3975` notifyMotion 入队；`:602` dispatchOnce → `:718` dispatchOnceInnerLocked）做焦点寻址与投递。**三线程=三道隔离舱**：读硬件的抖动、翻译的复杂度、分发的策略互不拖累。

5.2 通道与 App 侧逐段注释

java core/java/android/view/ViewRootImpl.java（骨架+讲解注释）

```
// —— 接收：WindowInputEventReceiver.onInputEvent（native 回调）——
// 讲解注释①：enqueueInputEvent 排队 → 若无更早未决事件立即处理
// —— doProcessInputEvents → deliverInputEvent ——
// 讲解注释②：InputStage 责任链依次过闸：
```

```
// NativeInputStage → ViewPreIme → ImeInputStage（输入法截胡）
// → ViewPostIme → SyntheticEvent（合成滚动惯性）
// → 最终 View.dispatchTouchEvent（你的 onTouchEvent 在此）
// ── finishInputEvent ──
// 讲解注释③：无论消费与否都要回执（未消费转给下一窗口/系统策略），
// dispatcher 记账并从 AnrTracker 摘除本事件——5s 闹钟解除。
```

逐段精读：②的责任链解释了"为什么输入法先于你的 onTouchEvent"——Ime 截胡是设计而非 bug。③的回执义务是 ANR 计时的正反面：**主线程卡住=回不来=闹钟响**（`AnrHelper.appNotResponding:71`，衔接第四册 §6）。

5.3 View 树事件分发逐段注释（面试永恒题，源码为准）

java core/java/android/view/ViewGroup.java dispatchTouchEvent（骨架+讲解注释）

```
public boolean dispatchTouchEvent(MotionEvent ev) {
// 讲解注释①：DOWN 起始：清旧目标；onInterceptTouchEvent 试拦截
// （仅父容器有此闸门——拦截即本容器作为普通 View 消费）。
// 讲解注释②：DOWN 命中测试：遍历子 View（逆 z 序，上层优先），
// 命中（坐标在其区域）则 child.dispatchTouchEvent 递归。
// 返回 true → 记 mFirstTouchTarget：后续 MOVE/UP 全归它
// ——这就是"滑动锁定"的机制（不再重新命中测试）。
// 讲解注释③：MOVE 期间 onInterceptTouchEvent 仍会问：
// 父容器可中途抢（子 View 想 Nesting：requestDisallowInterceptTouchEvent
// 是子视图"禁止父拦"的投票，DOWN 前设置无效的经典坑）。
// 讲解注释④：无人接 → 本容器 onTouchEvent（View 层）：
// OnTouchListener 优先于 onTouchEvent；点击长按即此层手势识别
// （View 的 onTouchEvent 内置 press/longpress/click 状态机）。
}
```

逐段精读四句话：**拦截权在父**（intercept）；**命中权在子**（hit test）；**锁定起于 DOWN**（firstTouchTarget）；**消费可中途易主**（onInterceptTouchEvent 每轮 MOVE 复议）。滑动冲突的官方答案（内外两层：同向拦截/异向投票）全部由这四条规则推导。

按键（key）链路差异要点：先过 `InputDispatcher` 拦截器（电源/音量系统级短路）→ 焦点窗口 → `ViewRootImpl.processKeyEvent` → Ime 截胡（书 §5.2）→ `dispatchKeyEvent` 自底向上（焦点 View 起，冒泡至 Activity）。**触摸是"命中"，按键是"焦点"**——两条寻址线在第六章合流。

5.4 触摸事件七段完整代码走读（教科书级逐步分解）

第①段：内核事件到达

text /dev/input/event3（触摸屏设备节点）

```
// 内核触摸驱动检测到手指按下 → 生成 input_event 结构
struct input_event {
struct timeval time; // 时间戳
__u16 type; // EV_ABS（绝对坐标）/ EV_SYN（同步）
__u16 code; // ABS_MT_POSITION_X / ABS_MT_POSITION_Y
__s32 value; // 坐标值
};
// 讲解注释①：一次触摸 = 多个事件（X坐标+Y坐标+压力+SYN同步标记）
// SYN 事件表示"这一批事件属于同一次触摸"
```

第②段：EventHub 聚合

cpp EventHub.cpp:1533 getEvents（骨架+注释）

```
// 讲解注释②：EventHub 用 epoll 监听所有 /dev/input/eventX
// 每个 fd 对应一个触摸屏/按键/传感器设备
// getEvents 返回 RawEvent[] 数组（多设备事件混合）
// inotify 监听设备插拔（新触摸屏连接 → 动态添加 fd）
```

第③段：InputReader 翻译

cpp InputReader.cpp:68-110（骨架+注释）

```
void InputReader::loopOnce() {
size_t count = mEventHub->getEvents(timeout, mEventBuffer, EVENT_BUFFER_SIZE);
processEventsLocked(mEventBuffer, count); // :110
// 讲解注释③：processEventsLocked 做三件事：
// a) 设备配置查询（分辨率/方向/触摸协议 A vs B）
// b) 坐标变换（内核原始坐标 → 显示坐标，含旋转/缩放）
// c) 手势合成（多指归并、点击/长按合成）
// 产出：NotifyMotionArgs（标准化事件）
}
```

第④段：InputDispatcher 选目标

cpp InputDispatcher.cpp:3975-718（骨架+注释）

```
void InputDispatcher::notifyMotion(const NotifyMotionArgs* args) {
// 讲解注释④：入队 → 按"焦点窗口"选目标
// a) 焦点窗口：WMS 告知哪个窗口有焦点（按键寻址）
// b) 触摸目标：按可见区域命中测试（多个窗口可命中=split touch）
// c) spy 窗口：无障碍/录屏等监控通道先过一道
}
int InputDispatcher::dispatchOnceInnerLocked(nsecs_t* nextWakeupTime) {
// 讲解注释⑤：从 mInboundQueue 取事件 → 按 Connection 分发
// 每个 Connection 对应一个 App 窗口的 InputChannel
// publishMotion → 写入目标 socket → App 的 Looper 唤醒
// ★ 此处跨进程：socket（非 Binder——高频小包场景）
}
```

第⑤段：InputChannel 传输

java InputChannel.java:109-118（骨架+注释）

```
// 讲解注释⑥：addWindow 时（WMS:1449）创建 socketpair：
// server 端 fd → 注册给 InputDispatcher（epoll 监听）
// client 端 fd → 跨进程传给 App（ParcelFileDescriptor）
// App 的 ViewRootImpl 把 client fd 挂到自己的 Looper
// → 事件到达时 Looper 唤醒 → WindowInputEventReceiver.onInputEvent
```

第⑥段：App 侧接收与分发

java ViewRootImpl.java（骨架+注释）

```
// WindowInputEventReceiver.onInputEvent → enqueueInputEvent
// → doProcessInputEvents → deliverInputEvent
```

```
// → InputStage 责任链：
// NativeInputStage → ViewPreIme → ImeInputStage（输入法截胡！）
// → ViewPostIme → SyntheticEvent（合成滚动）
// → View.dispatchTouchEvent（你的 onTouchEvent 在此）
// 讲解注释⑦：ImeInputStage 在 View 之前——
// 这就是"输入法先于你的 onTouchEvent"的机制层原因
```

第⑦段：消费与回执

java

```
// View.dispatchTouchEvent 返回 true → 已消费
// → finishInputEvent → socket 回执 → dispatcher 摘除 AnrTracker 计时
// 讲解注释⑧：无论消费与否都必须回执——
// 未消费 → dispatcher 转给下一个窗口
// 不回执 → 5 秒后 ANR！
```

七段全景图

text 触摸事件七段（标★=跨进程跳，标⑦=ANR计时点）

```
①内核驱动 ——→ ②EventHub(epoll) ——→ ③InputReader(翻译)
|
▼
④InputDispatcher(选目标)
|
★ socket
▼
⑤InputChannel ——→ ⑥ViewRootImpl(InputStage链) ——→ ⑦View.onTouchEvent
|
finishInputEvent ←————————┘
★ socket 回执（5s 超时=ANR）
```

设计洞察 · 逐段精读：为什么事件不走 Binder

①高频小包（200Hz 触摸流，每包几十字节）——Binder 每次事务有组包开销。②不能被线程池策略绑架——输入必须实时。③socket + epoll 是 Linux 最成熟的实时 IPC——选它是工程成熟度的体现。**"为每个场景选对 IPC"是系统开发者的基本功。**

第六章 焦点、触摸目标与多屏

设计洞察 · 学习目标

1. 区分焦点（按键寻址）与触摸目标（命中测试）两套逻辑；2. 解释 per-display 独立焦点与多屏窗口的关系；3. 用"点击穿透"现象反推 hit test 的候选顺序；4. 用 dumpsys input 还原一次分发

窗口**焦点**（key 事件寻址）与**触摸目标**（motion 事件命中）是两套独立逻辑：焦点由 WMS 依层级与 token 归属授予（`InputDispatcher` 的 focusedWindow），触摸则按可见区域命中测试（可分发给多个窗口形成 split touch）。多屏（折叠/外接）下每块屏一个 `DisplayContent`，焦点 per-display 独立——多任务拖拽跨屏的底层即此。面试点睛：**点击穿透不是"没接到"，是 hit test 找**

了别人；`dumpsys input` 的 touch 流水可还原每次分发。

第七章 实战：一次掉帧排查全记录

> **设计洞察 · 学习目标**
> 1. 背出五步法：定界→分责→定位→归因→复验；2. 用 gfxinfo/framestats 确认掉帧存在；3. 用 Perfetto FrameTimeline 读写责任标签；4. 把一次 measure 慢的案例走完闭环

假设现象：列表快速滑动偶发掉帧。标准流程五步：

- **第一步 定界**：perfetto 抓 5s trace；FrameTimeline 定位 jank 帧（§4.3 表）。
- **第二步 分责**：Jank(App) → 下钻 UI 轨道：`Choreographer#doFrame` 分段显示 traversal 18ms（预算 16.6ms）。
- **第三步 定位**：展开 traversal：`measure/layout` 12ms 异常（正常 2-3ms）——ListView 某行 wrap_content 触发整列重测。
- **第四步 归因**：对照 §3.3 两次 measure 之谜 → 定宽行容器解决。
- **第五步 复验**：重抓 trace，FrameTimeline 全绿。

这套"定界→分责→定位→归因→复验"是所有系统性能问题的通用方法论，比记住任何单个结论都值钱。**trace 不会说谎，但读 trace 的人会看错段落**——本册的所有图与代码，最终都是为了让你在 trace 里认出自己读过的每一个函数名。

第八章 深水区：交互系统的四条支线

> **设计洞察 · 学习目标**
> 1. 解释按键拦截链两道闸与"App 只是 fallback"；2. 对比手势导航的事件通知与预测返回的过程协商；3. 说明 FMQ 为何替代 Binder 承载高频传感器数据；4. 概述 IMMS 绑定模型与 InputConnection 文本协议的卡顿根源

8.1 按键拦截链：系统级短路

按键两道闸：入队前 `interceptKeyBeforeQueueing`（`InputDispatcher.cpp:3935` → `InputManagerService.java:3003` → WMS InputPolicy）与分发前 interceptKeyBeforeDispatching。电源/音量组合（截图/复位）在策略层直接消费并标记不再分发——**App 永远收不到原始事件**。经典题解：通话里音量键是音量、相机里是快门，因为拦截链给系统服务优先解释权，App 只是 fallback。

8.2 手势导航与预测性返回

手势导航=手势区命中后由系统**合成 KEYCODE_BACK**（衔接 B.1 按键链）。预测性返回（13 opt-in）：`core/java/android/window/OnBackInvokedDispatcher.java:28` 新回调体系——手势按下即发渐进回调（progress/start/cancel），App 实时预览可取消，系统侧同播手势动画；**从"事件通知"进化为"过程协商"**。未适配应用自动降级旧 KeyEvent。

8.3 传感器：FMQ 直通道与融合

`frameworks/native/services/sensorservice/`：SensorService 直持 HAL，数据面走 **FMQ 环形队列**（AIDL `hardware/interfaces/sensors/aidl/.../ISensors.aidl:129-134` EventQueue+旗标位）——共享内存批量收割，绕开 binder；控制面（启停/配置）才走 binder。服务侧做融合（Fusion.cpp：陀螺+磁+加→姿态）与批处理；direct channel 更进一步（HAL/DSP 直写共享内存，App 零拷贝读）。**高频小包走 FMQ** 与 Provider 的 CursorWindow 是同一哲学的两次应用。

8.4 IME：文字编辑的隐形协议

`services/core/.../inputmethod/`：IMMS 管输入法生命周期（InputMethodBindingController 绑定/重连）、焦点与显示位（Insets 协商）。暗线是 **InputConnection 文本协议**：commitText/getTextBeforeCursor 每次一条 binder 往返——输入法每键多次查询即卡顿根源；优化靠批量编辑与少查上下文。13 新增手写就地输入（HandwritingEventReceiverSurface）把笔迹 Surface 直接递给 IME，老协议的新血。

8.5 无障碍框架

`frameworks/base/services/accessibility/`：AccessibilityManagerService 管服务绑定与事件过滤；事件流=View 树生成 AccessibilityNodeInfo 快照→有订阅才序列化分发；**可注入操作**（GestureDescription 合成触摸）走 dispatcher 合成入口。安全敏感：a11y 可读屏可操作，13 起强制显式授权与来源限制——机制与伦理同框。

> **源码锚点 · 本章锚点**
> InputDispatcher.cpp:3935 / InputManagerService.java:3003 拦截链桥接
window/OnBackInvokedDispatcher.java:28 预测返回
sensorservice/{Fusion,AidlSensorHalWrapper}.cpp 与 ISensors.aidl:129 FMQ
inputmethod/InputMethodBindingController.java / services/accessibility/

第九章 渲染衔接：RenderThread 与 SurfaceFlinger

> **设计洞察 · 学习目标**
> 1. 说出一帧后半程：DisplayList→RenderThread→BufferQueue→SF；2. 解释 RenderNode 属性动画为何不重录；3. 说明 Overlay 直通的省电原理；4. 划清本册与渲染专册的接口线

主线程录完 DisplayList 后的接力棒：

```text 硬件加速一帧的后半程（libhwui/SF 视角）
ViewRootImpl.draw → RecordingCanvas 记命令树（DisplayList）
→ RenderThread：syncAndDrawFrame（与 UI 线程同步内容）
→ ① 渲染管线：OpenGL/Vulkan 后端执行 DisplayList（Skia）
→ ② 纹理与缓存：RenderNode 属性（位移/透明度）可直接改不重录
→ ③ 提交：queueBuffer 入 BufferQueue（三缓冲轮转）
→ SurfaceFlinger（下一个 SF VSync）：acquire→合成（HWComposer Overlay/
GPU 合成两路）→ present 到面板
```

三处读码价值最高的点：**RenderNode 属性动画**（View.setTranslationX 不触发重录，仅改节点矩阵——属性动画流畅的秘密）；**Overlay 直通**（视频层常绕过 GPU 合成直叠面板，省电大项）；**帧时间线**（FrameTimeline 把 expected/actual 呈现时刻逐帧对账，掉帧责任链完整可查）。至此第五册与 SurfaceFlinger 世界的接口线画完——更深的图形栈留给后续渲染专册。

6.1 split-touch 与多指细节

MotionEvent.ACTION_POINTER_DOWN/UP 把多指编码进同一事件流（pointer index/位图）；**每根手指可以命 中不同窗口**（splitTouch=true 时 dispatcher 把一个 MotionEvent 拆成多路独立投递）——双人同屏游戏与输入法+手写并行都依赖它。ACTION_CANCEL 则是系统"强制收摊"：窗口失焦/被遮挡时给在途序列一个终结标记（**收到 CANCEL 必须清理按下态**，否则按钮永久高亮这类诡异 bug 就是它）。

7.2 掉帧排查完整教程（教科书级五步法）

第一步：确认有掉帧（不是"感觉卡"）

```bash
# 方法 A：dumpsys 帧统计
adb shell dumpsys gfxinfo com.example.app
# 看最后两行：
# Total frames rendered: 1200
# Janky frames: 45 (3.75%) ← 超过 1% 就值得关注
# 90th percentile: 16ms ← 90% 的帧在 16ms 内完成
# 方法 B：实时帧率
adb shell dumpsys gfxinfo com.example.app framestats
# 输出每帧的详细时间戳（FrameTimeline 数据）
```

第二步：用 Perfetto 定位掉帧位置

bash

```
# 抓 5 秒 trace
adb shell perfetto -c - --txt -o /data/misc/perfetto-traces/jank.pftrace << 'EOF'
buffers { size_kb: 65536 }
data_sources { config { ftrace_config {
ftrace_events: "sched/sched_switch"
} } }
data_sources { config {
android_buffers_track { }
} }
duration_ms: 5000
EOF
adb pull /data/misc/perfetto-traces/jank.pftrace
# 在 https://ui.perfetto.dev 打开
```

第三步：读 FrameTimeline 责任段

text Perfetto FrameTimeline 中的掉帧标注

```
帧号 期望呈现 实际呈现 责任 原因
4812 16.6ms 16.6ms None 正常
4813 33.3ms 50.0ms Jank(App) ← App 的 traversal 超时
4814 50.0ms 50.0ms None 追帧成功
4815 66.6ms 100.0ms Jank(SF) ← SF 合成超时
4816 83.3ms 100.0ms Jank(disp) ← 显示面板延迟
```

第四步：下钻定位慢在哪一段

责任段	可能原因	下钻方法
Jank(App)	traversal 超 16ms	展开 doFrame 轨道：measure/layout/draw 哪个慢
Jank(RT)	RenderThread 慢	展开 GPU 轨道：着色器编译/纹理上传/大 canvas
Jank(SF)	SF 合成慢	展开 SF 轨道：HWC validate/present 哪个卡
Jank(disp)	面板延迟	检查刷新率是否切换中

第五步：定向优化

瓶颈段	优化方案	预期效果
measure/layout 慢	减少嵌套层次/避免 wrap_content 双测量	减 5-15ms
draw 慢（软件）	开启硬件加速/减少过度绘制	减 10-30ms
RenderThread 慢	减少过度绘制/简化着色器	减 5-20ms
频繁 GC	减少 onDraw 中的对象分配	消除 GC 暂停

瓶颈段	优化方案	预期效果
SF 合成慢	减少窗口层数/使用 Overlay 直通	减 5-10ms

一个真实案例的完整分析

text 场景：列表滚动偶发掉帧（Janky frames: 8%）

```
FrameTimeline 分析 → Jank(App) 占 95%
下钻 doFrame 轨道：
measure: 12ms（正常 2-3ms） ← 异常!
layout: 3ms（正常）
draw: 5ms（正常）
分析 measure 慢的原因：
→ 展开 View 层级 → 发现某行 item 是嵌套 5 层的 LinearLayout
→ 每层都是 wrap_content → 触发双测量
→ 改为 ConstraintLayout（单层）+ 固定高度
→ 复测：measure 降至 3ms，Janky frames: 0.8%
// 讲解注释：这就是"度量→分析→优化→复验"的完整闭环
```

设计洞察 · 逐段精读：为什么用 Perfetto 不用 systrace

systrace 已废弃（Python 2 依赖）；Perfetto 是继任者（C++/Web UI/云同步）。**工具会换代，但"度量→分析→优化→复验"的方法论永远不过时**。

第十章 设计决策八问

设计洞察 · 学习目标

1. 复述八个"为什么"并给出机制依据；2. 把 SurfaceView/TextureView 的取舍讲成缓冲所有权问题；3. 把属性动画接入 Choreographer 车道并解释"动画先于遍历"；4. 面对开放设计题时套用"职责分离+时序窗口"的分析框架

9.1 补全：SurfaceView 与动画体系的两条接入线

SurfaceView（`core/java/android/view/SurfaceView.java:239` 构造族）：不走 View 树常规绘制——它向 WMS 申请**独立 Surface**（updateSurface 路径，:154-160），在窗口上"打洞"让下层 Surface 透出（或 setZOrderedOnTop 浮于其上）。视频/相机预览/游戏选它的理由：**独立缓冲独立节拍**（视频帧率不受 UI 帧率绑架、GPU 上屏路径更短）。代价：与 View 动画体系天然不咬合（13 起用 BLASTBufferQueue 与控制流更好协调，但仍非同窗）。TextureView 是反面：内容画进 View 树的 Surface 内（共享缓冲、可动画），代价是多一次合成与 GPU 拷贝。

动画的两条接入线：View 动画（老）只影响绘制矩阵（draw 阶段变换，**属性没变**——经典"动画完跳回"问题的根源）；属性动画（ValueAnimator/ObjectAnimator）经 **AnimationHandler**（`core/java/android/animation/AnimationHandler.java:24/:83` 的 FrameCallback）挂进 **Choreographer 的 CALLBACK_ANIMATION 车道**——每个 vsync 算一帧插值、改真实属性、触发

invalidate 走 TRAVERSAL 车道。对照书5 §4.1 的四车道顺序：**动画先于遍历**正是为了"本帧算值、本帧画完"。RenderThread 侧的动画（书5 §8 RenderNode 属性直改）则连 invalidate 都不必——流畅的最后一公里。

- **为什么事件先过 system_server 再到 App?** 焦点仲裁、系统键拦截、ANR 统一计时都需要全局视角；直连方案省不了多少延迟，却失去全部治理能力。
- **为什么用 socketpair 而不是 Binder 传输入?** 输入是高频小包+fd 复用场景；且分发线程不能被 binder 线程池策略绑架。
- **为什么 Choreographer 四车道固定顺序?** 同帧"输入→动画→绘制"因果闭环，乱序即恒定一帧触控延迟。
- **为什么同步屏障合法?** 帧的时效性高于普通消息公平性；屏障只存在于 vsync 唤醒到 traversal 完成的窗口期。
- **为什么三缓冲不多不少?** 双缓冲互等、三缓冲容忍一抖、四缓冲纯增延迟——内存与流畅的帕累托前沿。
- **为什么 WMS 不碰像素?** 职责单一+进程隔离：户口、层位、动画编排与内容生产、合成彻底解耦（崩溃域与性能域同时隔离）。
- **为什么硬件加速先录 DisplayList?** 录制（UI 线程）与执行（渲染线程）解耦；局部 invalidate 只重录子树重放整树，均摊成本。
- **为什么输入 ANR 只有 5s?** 输入卡顿直接被手指感知；广播/服务是异步语义可宽容——超时线差异即"用户在场感"差异。

附录 A 观测实操

目标	命令/工具	看点
窗口全景	dumpsys window windows	WindowState/layer/token/可见性
输入全景	dumpsys input	设备、通道、焦点、touch 流水、ANR 计时
裸事件	getevent /dev/input/eventX	不经 framework 的硬件层定界
帧时序	perfetto（FrameTimeline）/ dumpsys SurfaceFlinger --latency	掉帧责任分段
UI 线程	debuggerd -b	traversal/input 栈型
屏幕层级	dumpsys window displays	DisplayContent/Insets 分区

附录 B 自检题

- 画出触摸事件从内核到 onTouchEvent 的完整链路，标注每次线程/进程切换。
- 同步屏障何时插入何时移除？拿掉它帧率会怎样？
- 三幕剧中哪一幕禁止改尺寸？为什么？
- 焦点与触摸目标的差异举一个现象级例子。
- addWindow 失败的两大类原因分别对应什么时序？
- 三缓冲下"App 晚了一帧"为什么用户看到的不是黑屏？

附录 C 快问快答（面试扫雷 20 条）

- Window/PhoneWindow/DecorView/ViewRootImpl 关系？——壳/唯一实现/树根/窗口总管四层。
- BadTokenException 何时发生？——token 未注册（对话框早于 attach）或已失效。
- 同步屏障谁插谁拔？——scheduleTraversals 插，doTraversal 拔；只服务 vsync 消息插队。
- Choreographer 四车道顺序？——INPUT→ANIMATION→INSETS→TRAVERSAL（:226/:256）。
- invalidate 与 postInvalidate 区别？——同帧合并欠条；后者线程安全（跨线程投递）。
- requestLayout 冒泡到哪停？——ViewRootImpl（assignParent 链的根）。
- 一帧两次 measure 常见原因？——wrap_content/weight/依赖测量。
- 硬件加速下画在哪执行？——UI 线程录、RenderThread 执行（双时钟 getSfInstance:925）。
- setTranslationX 为什么不重绘？——RenderNode 属性，改矩阵不重录 DisplayList。
- 触摸事件 window 间分发顺序？——命中测试（可 split）；key 走焦点链。
- requestDisallowInterceptTouchEvent 为何 DOWN 前无效？——DOWN 时刻清目标与投票本。
- 输入 ANR 计时起点？——dispatcher publishMotion 时刻（AnrTracker 记账）。
- getevent 看得到就说明什么？——内核与设备正常，问题在 framework 及以上。
- dumpsys input 里 focusedWindow 空？——窗口未获焦（token/层序/启动态问题）。
- 三缓冲谁分配？——BufferQueue（GraphicAlloc），槽位经 SF 与生产者协商。
- Overlay 合成的条件？——层不混合/无缩放/格式匹配，HWComposer 决策。
- 掉帧一定是 App 慢吗？——FrameTimeline 分责：App/RenderThread/SF/Disp 四段。
- 多屏焦点？——per-DisplayContent 独立焦点窗口。
- 无障碍会影响输入吗？——AccessibilityInputFilter 是责任链一环（事件可被改写/吞掉）。
- 本册之后读什么？——第六册（包从哪来）与渲染专册（BufferQueue/BLAST）。

第十一章 同步屏障深潜：一帧的优先级引擎

> **设计洞察 · 学习目标**
>
> 1. 说出同步屏障的物理形态：一条 target 为 null 的 Message；2. 逐行解释 postSyncBarrier、removeSyncBarrier、next() 三段真实源码；3. 解释 vsync 消息为何必须 setAsynchronous(true) 以及屏障插入时为何绝不唤醒队列；4. 用"悬空屏障"现象解释主线程假死的队列侧成因与验证思路

11.1 本章在链路中的位置

§3.1 的欠条模型在第二步埋了个伏笔：`scheduleTraversals` 调 `postSyncBarrier` "挡住普通消息，让 vsync 插队"；§4.1 讲了 vsync 抵达后 doFrame 四车道发车。但**中间那根接力棒——vsync 消息如何带着"异步"标记穿过屏障、队列如何在屏障之下维持其他消息的公平——两章都没有拆开**。本章拆它。链条横跨两个文件：`MessageQueue.java`（队列引擎）与 `Choreographer.java`（vsync 接收端）。读完你应能回答本章标题背后最阴的面试题：**同步屏障插着不拔，主线程会怎样?**

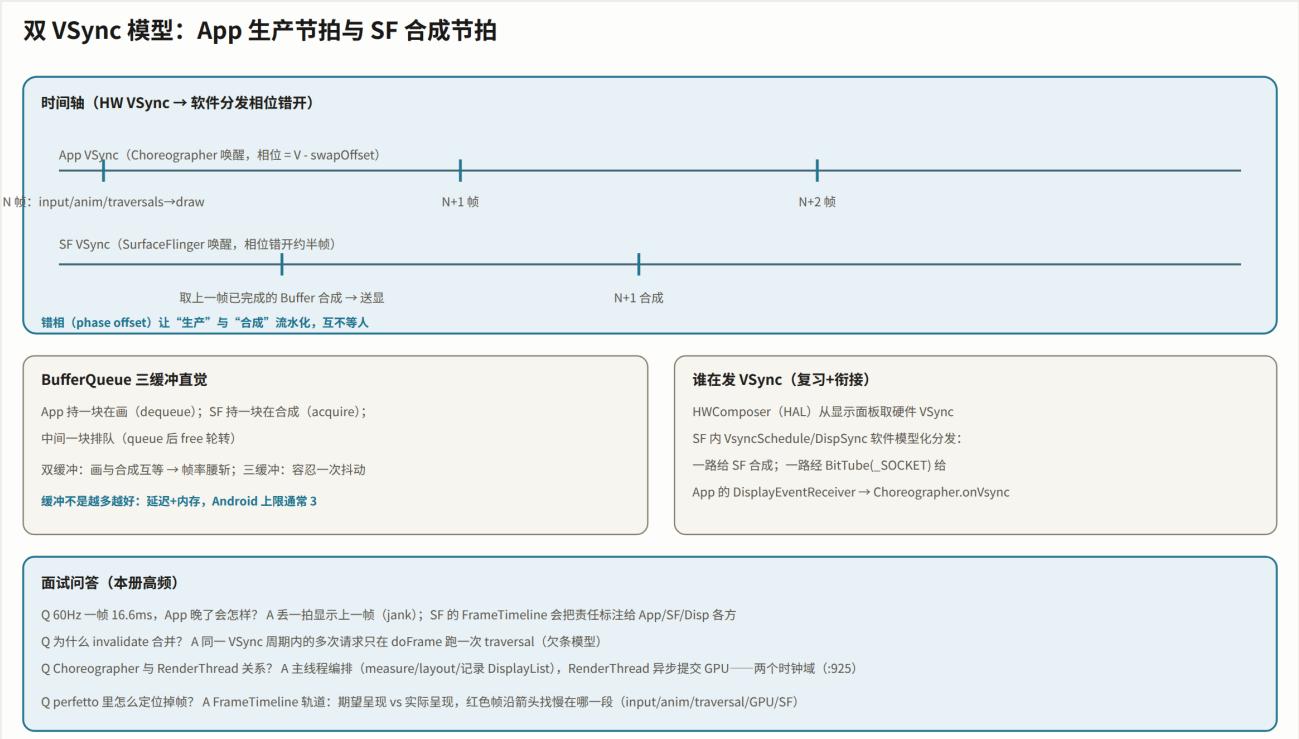


图 6 双 VSync 相位错开模型、BufferQueue 三缓冲与掉帧责任判定（App 侧时间线即本章的队列世界）

11.2 屏障的插与拔：MessageQueue 逐行

先纠正一个流行误解：**同步屏障不是队列的开关，而是一条特殊的 Message**——它 `target == null`（普通消息必须指向发送它的 Handler），仅凭这一点，它出现在队头时整个取数逻辑就改写了。

java frameworks/base/core/java/android/os/MessageQueue.java:476-549（插与拔，真实代码+讲解注释）

```
private int postSyncBarrier(long when) {
```

```
// Enqueue a new sync barrier token.
// We don't need to wake the queue because the purpose of a barrier is to stall it.
// 讲解注释①：源码注释即天机——屏障的使命是"堵"，因此插入时绝不 nativeWake。
// 主线程此刻若睡在 nativePollOnce 上会继续睡，直到下一个"被允许"
// 的消息（异步消息或定时到期）把它叫醒。
synchronized (this) {
final int token = mNextBarrierToken++;
final Message msg = Message.obtain();
msg.markInUse();
msg.when = when;
msg.arg1 = token; // 讲解注释②：token 存进 arg1，
// removeSyncBarrier 靠它认领这条屏障
Message prev = null;
Message p = mMessages;
if (when != 0) {
while (p != null && p.when <= when) {
prev = p;
p = p.next;
}
}
if (prev != null) { // invariant: p == prev.next
msg.next = p;
prev.next = msg; // 讲解注释③：按 when 有序插入——
// 屏障也是普通链表节点
} else {
msg.next = p;
mMessages = msg; // 讲解注释④：最常见情形：插到队头
}
return token;
}
}
public void removeSyncBarrier(int token) {
// Remove a sync barrier token from the queue.
// If the queue is no longer stalled by a barrier then wake it.
synchronized (this) {
Message prev = null;
Message p = mMessages;
while (p != null && (p.target != null || p.arg1 != token)) {
prev = p; // 讲解注释⑤：沿链表寻找 target==null
p = p.next; // 且 arg1==token 的那条屏障
}
if (p == null) {
throw new IllegalStateException("The specified message queue synchronization "
+ " barrier token has not been posted or has already been removed.");
// 讲解注释⑥：找不到=有人忘了拔——框架
// 用异常把"漏账"变成显式崩溃而非静默卡死
}
final boolean needWake;
if (prev != null) {
prev.next = p.next;
needWake = false;
} else {
mMessages = p.next;
needWake = mMessages == null || mMessages.target != null;
} // 讲解注释⑦：仅当拔的是队头屏障、且
```

```
// 后面还有被憋住的同步消息才唤醒
p.recycleUnchecked();
if (needWake && !mQuitting) {
nativeWake(mPtr);
}
}
}
```

设计洞察 · 逐段精读：插入不唤醒、拔除才可能唤醒

①与⑦是一对镜像设计：插屏障时主线程多半正忙着"欠账"（scheduleTraversals 在遍历前的窗口期调用它），接下来本线程自己就会处理 vsync，不需要唤醒；拔屏障时若队里憋着普通消息（它们排在屏障之后、异步消息之前），必须 nativeWake 让 Looper 回来收割——否则它们要等下一次任意队列事件才被执行。"堵"与"放"的唤醒策略相反，说明屏障本质是"临时改写队头语义"，而非队列状态开关。

11.3 队列在屏障之下：next() 逐行

屏障插上之后，Looper 的每一次取消息都经过这段代码（Android 中被面试背诵最多、被读原文最少的函数）：

java frameworks/base/core/java/android/os/MessageQueue.java:319-368（next() 取数循环，真实代码+讲解注释）

```
Message next() {
final long ptr = mPtr;
if (ptr == 0) {
return null; // 讲解注释①：队列已销毁（quit 过）
}
int pendingIdleHandlerCount = -1;
int nextPollTimeoutMillis = 0;
for (;;) {
if (nextPollTimeoutMillis != 0) {
Binder.flushPendingCommands();
}
nativePollOnce(ptr, nextPollTimeoutMillis);
// 讲解注释②：睡眠/等待：无消息则阻塞，
// 有定时消息则睡到 msg.when
synchronized (this) {
final long now = SystemClock.uptimeMillis();
Message prevMsg = null;
Message msg = mMessages;
if (msg != null && msg.target == null) {
// Stalled by a barrier. Find the next asynchronous message in the queue.
do {
prevMsg = msg;
msg = msg.next;
} while (msg != null && !msg.isAsynchronous());
}
// 讲解注释③：全章题眼——队头 target==null
// 即屏障在岗：跳过所有同步消息，
// 只找第一条异步消息
if (msg != null) {
if (now < msg.when) {
nextPollTimeoutMillis = (int) Math.min(msg.when - now, Integer.MAX_VALUE);
```

```
// 讲解注释④：还没到期，睡到到期为止
} else {
mBlocked = false;
if (prevMsg != null) {
prevMsg.next = msg.next;
} else {
mMessages = msg.next;
}
msg.next = null;
msg.markInUse();
return msg;
// 讲解注释⑤：摘下的是异步消息本身；
// 屏障与被拦的同步消息原地不动
}
} else {
nextPollTimeoutMillis = -1;
} // 讲解注释⑥：屏障+无异步消息=无限睡——
// 这就是"悬空屏障"假死的机制
```

屏障之下还有一条隐藏规则在**入队侧**：新消息到达时要不要唤醒队列？

java frameworks/base/core/java/android/os/MessageQueue.java:548-596（enqueueMessage 唤醒判定，节选+讲解注释）

```
boolean enqueueMessage(Message msg, long when) {
if (msg.target == null) {
throw new IllegalArgumentException("Message must have a target.");
}
// 讲解注释①：公开 API 发不出"屏障"——
// 屏障只能由 postSyncBarrier 这类系统
// 内部入口制造
synchronized (this) {
...
msg.markInUse();
msg.when = when;
Message p = mMessages;
boolean needWake;
if (p == null || when == 0 || when < p.when) {
// New head, wake up the event queue if blocked.
msg.next = p;
mMessages = msg;
needWake = mBlocked; // 讲解注释②：成为新队头且主线程在睡 → 唤醒
} else {
// Inserted within the middle of the queue. Usually we don't have to wake
// up the event queue unless there is a barrier at the head of the queue
// and the message is the earliest asynchronous message in the queue.
needWake = mBlocked && p.target == null && msg.isAsynchronous();
// 讲解注释③：中间插入通常不唤醒——除非
// 队头有屏障且本消息是异步的：
// 这正是 vsync 消息到达时的唤醒路径
...
}
if (needWake) {
nativeWake(mPtr);
}
}
```

```
return true;
}
```

设计洞察 · 设计洞察：公平让位于帧时效

普通消息按 when 先来先服务；屏障一插，同步消息整体降权。这是 Android 写进注释的取舍（"Usually we don't have to wake up..."）——帧的时效性高于消息公平性。呼应第十章"为什么同步屏障合法"：它只在 vsync 唤醒到 traversal 完成的窗口期存在，代价有界。调试时若发现 mTraversalScheduled 为 true 却长期不遍历，第一个怀疑对象就是悬空屏障。

11.4 异步消息的出生地：FrameDisplayEventReceiver.onVsync

屏障要拦的是同步消息，那么谁被授予"异步"通行证？Choreographer 自产的消息**全部**打了异步标记：MSG_DO_SCHEDULE_CALLBACK（:488）、MSG_DO_SCHEDULE_VSYNC（:692）、MSG_DO_FRAME（:702），以及本节的主角——vsync 到达后重投的消息（:1204）。

java frameworks/base/core/java/android/view/Choreographer.java:1172-1214（onVsync 与 run，真实代码+讲解注释）

```
@Override
public void onVsync(long timestampNanos, long physicalDisplayId, int frame,
VsyncEventData vsyncEventData) {
try {
if (Trace.isTagEnabled(Trace.TRACE_TAG_VIEW)) {
Trace.traceBegin(Trace.TRACE_TAG_VIEW,
"Choreographer#onVsync "
+ vsyncEventData.preferredFrameTimeline().vsyncId);
}
// Post the vsync event to the Handler.
// The idea is to prevent incoming vsync events from completely starving
// the message queue. ...
long now = System.nanoTime();
if (timestampNanos > now) {
Log.w(TAG, "Frame time is " + ((timestampNanos - now) * 0.000001f)
+ " ms in the future! Check that graphics HAL is generating vsync "
+ "timestamps using the correct timebase.");
timestampNanos = now;
}
// 讲解注释①：对表：硬件时间戳若晚于本机时钟
// 说明 HAL 时间基错了——先钳制再继续
if (mHavePendingVsync) {
Log.w(TAG, "Already have a pending vsync event. There should only be "
+ "one at a time.");
} else {
mHavePendingVsync = true;
}
// 讲解注释②：同屏只允许一张在途 vsync 票
mTimestampNanos = timestampNanos;
mFrame = frame;
mLastVsyncEventData = vsyncEventData;
Message msg = Message.obtain(mHandler, this);
msg.setAsynchronous(true);
// 讲解注释③：全章题眼——这条消息被标记为
```

```
// 异步，才能穿过 scheduleTraversals 在
// ViewRootImpl.java:2260 插下的屏障
mHandler.sendMessageAtTime(msg, timestampNanos / TimeUtils.NANOS_PER_MS);
// 讲解注释④：按 vsync 时刻定时投递——
// 早到的回调也等到帧时刻才执行
} finally {
Trace.traceEnd(Trace.TRACE_TAG_VIEW);
}
}
@Override
public void run() {
mHavePendingVsync = false;
// 讲解注释⑤：票据回收：允许接收下一张
doFrame(mTimestampNanos, mFrame, mLastVsyncEventData);
// → §4.1 四车道开演
}
```

设计洞察 · 设计洞察：为什么 onVsync 不直接调 doFrame

源注释给了答案：vsync 事件由 native Looper 回调送达，若就地执行 doFrame，它会插队到所有早于帧时刻的普通消息之前——vsync 风暴会饿死队列。重投一条 sendMessageAtTime(timestamp) 的异步消息后，doFrame 恢复按时间戳排队：与同步消息比，它凭"异步"越过屏障；与其他消息比，它按 when 公平竞争。**一次重投同时解决插队与公平两个问题**——这是本册最精巧的四行代码。

11.5 悬空屏障：主线程假死的队列侧解剖

把三段代码合起来看，故障模式就清晰了：**postSyncBarrier 与 removeSyncBarrier 必须成对出现**。本册已见的配对是 `scheduleTraversals`（:2260 插）/ `doTraversal`（:2280 拔），`unscheduleTraversals`（:2271）也会拔。若任何路径漏拔（历史版本真出过这类 bug），next() 在讲解注释③的 do-while 里永远找不到异步消息，主线程无限睡在讲解注释⑥——现象是**应用完全冻结但进程存活、内存正常、Binder 线程健康**。

验证思路（不进调试器）：`debuggerd -b <pid>` 抓全部线程栈——主线程若停在 `MessageQueue.next → nativePollOnce`，配合 logcat 中 `Skipped N frames` 持续增长（Choreographer.java 的 skippedFrames 日志，§4.1 讲解注释①的同源逻辑），即可归因队列侧。这类问题几乎都在框架/魔改 ROM 侧，应用侧能做的是提供复现路径与 trace。注意与第八章 ANR（5 秒回执）区分：悬空屏障卡的是**整条主线程**，输入回执自然也回不来，所以它最终会以 ANR 面目出现在用户面前。

源码锚点 · 第十一章锚点

MessageQueue.java:319 next / :476 postSyncBarrier / :517 removeSyncBarrier / :548 enqueueMessage（异步唤醒 :580）
Choreographer.java:1172 onVsync（:1204 setAsynchronous）/ :1210 run；MSG_* 异步标记 :488/:692/:702
ViewRootImpl.java:2257 scheduleTraversals（:2260 插屏障）/ :2268 unscheduleTraversals / :2277 doTraversal（:2280 拔屏障）

第十二章 WindowState 的一生：出生、装户与死亡清理

设计洞察 · 学习目标

1. 逐段读懂 WindowState 构造函数：InputWindowHandle 装配、linkToDeath、基线层位计算、WindowStateAnimator 挂载；2. 说出 removeIfPossible 的四条分支：替换保活/死亡保活/退场动画/立即移除；3. 列出 removeImmediately 的清理清单并说明各自防止哪类泄漏；4. 用 WIN DEATH 与 LEAK 日志归因 App 进程死亡后的窗口清理路径

12.1 本章在链路中的位置

§1.3 的 addWindow 审验漏斗四步——屏→token→权限→建籍——其中讲解注释④"new WindowState（一切窗口事实的载体）"只有一行。这一行背后是 WMS 侧最重的构造函数之一：**一个 WindowState 出生时就要装配输入户籍（第十三章的素材）、注册死亡讣告、算出层位基线（§1.2 z 序的素材）、挂上动画伴随者**。而"建籍"的终点 removeIfPossible 同样不是一行 free——它是四条分支的清算庭。本章读完，addWindow 与 removeWindow 两端都闭环。

12.2 出生：WindowState 构造函数逐段

java frameworks/base/services/core/java/com/android/server/wm/WindowState.java:1120-1254（构造，骨架+讲解注释）

```
WindowState(WindowManagerService service, Session s, IWindow c, WindowToken token,
WindowState parentWindow, int appOp, WindowManager.LayoutParams a, int viewVisibility,
int ownerId, int showUserId, boolean ownerCanAddInternalSystemWindow,
PowerManagerWrapper powerManagerWrapper) {
super(service);
...
mSession = s;
mClient = c; // 讲解注释①：s=跨进程 Session（每 App 一条，
// §2.1 注释④）；c=App 侧 IWindow 存根——
// 死亡讣告的监听对象
mToken = token;
mActivityRecord = mToken.asActivityRecord();
// 讲解注释②：token 多态解包：应用 token 能解出
// ActivityRecord，系统 token 得 null——
```

```
// §1.1 "token 即户口"的类型级体现
...
mWindowId = new WindowId(this);
mAttrs.copyFrom(a);
...
DeathRecipient deathRecipient = new DeathRecipient();
// 讲解注释③：创建讣告接收者（:3120）——
// App 进程崩溃时 binderDied 从这里接手
mInputWindowHandle = new InputWindowHandleWrapper(new InputWindowHandle(
mActivityRecord != null
? mActivityRecord.getInputApplicationHandle(false) : null,
getDisplayId()));
mInputWindowHandle.setFocusable(false);
mInputWindowHandle.setOwnerPid(s.mPid);
mInputWindowHandle.setOwnerUid(s.mUid);
mInputWindowHandle.setName(getName());
// 讲解注释④：出生即装配输入户籍：pid/uid/包名/
// 类型一次填齐——第十三章 updateInputWindowsLw
// 上报给 InputDispatcher 的就是这些句柄
mInputWindowHandle.setTrustedOverlay(
((mAttrs.privateFlags & PRIVATE_FLAG_TRUSTED_OVERLAY) != 0
&& mOwnerCanAddInternalSystemWindow)
|| InputMonitor.isTrustedOverlay(mAttrs.type));
// 讲解注释⑤：受信 overlay 白名单：决定合成侧
// 是否采信该窗口的触控遮挡声明
try {
c.asBinder().linkToDeath(deathRecipient, 0);
} catch (RemoteException e) {
mDeathRecipient = null;
...
mWinAnimator = null;
...
return;
}
// 讲解注释⑥：注册讣告失败（对端已死）→
// 半初始化即返回：mWinAnimator 为 null，
// 后续清理路径必须容忍这种"残躯"
if (mAttrs.type >= FIRST_SUB_WINDOW && mAttrs.type <= LAST_SUB_WINDOW) {
mBaseLayer = mPolicy.getWindowLayerLw(parentWindow)
* TYPE_LAYER_MULTIPLIER + TYPE_LAYER_OFFSET;
mSubLayer = mPolicy.getSubWindowLayerFromTypeLw(a.type);
mIsChildWindow = true;
// 讲解注释⑦：子窗口层位=父窗口基线*倍率+偏移；
// mSubLayer 决定同父下的上下——§1.2 的实现
} else {
mBaseLayer = mPolicy.getWindowLayerLw(this)
* TYPE_LAYER_MULTIPLIER + TYPE_LAYER_OFFSET;
mSubLayer = 0;
...
}
mIsFloatingLayer = mIsImWindow || mIsWallpaper;
// 讲解注释⑧：IME 与壁纸是"浮层"：布局时用
// 特殊坐标系（不占应用可用区）
...
mWinAnimator = new WindowStateAnimator(this);
```

```
mWinAnimator.mAlpha = a.alpha;
// 讲解注释⑨：动画伴随者上场：Surface 的创建/
// 销毁与窗口动画都由它执行——§12.3 清理清单
// 的第一步就是它的 destroySurfaceLocked
if (mIsChildWindow) {
parentWindow.addChild(this, sWindowSubLayerComparator);
} // 讲解注释⑩：子窗口挂父树；源码注释强调
// "先初始化完所有字段再加进父窗，防止
// onDisplayChanged 期间异常"——顺序即安全
}
```

设计洞察 · 逐段精读：为什么构造函数里就要 linkToDeath

addWindow 是跨进程调用，App 端在拿到返回值之前就可能死亡（低内存被杀）。若 WMS 等"下一次操作"才发现对端已死，窗口会带着有效 Surface 与输入通道在系统里存活一段无人认领的时间——这正是经典 leaked surface 的温床。linkToDeath 把"对端死亡"变成同步事件：binder 驱动保证讣告投递，binderDied（:3122）随即进入 §12.4 的清理。**生命周期管理与 Binder 死亡通知绑定，是 system_server 服务的通用范式**（书3 的服务死亡重绑同理）。

12.3 死亡并不简单：removeIfPossible 的四条分支

App 调 removeView（或系统侧 token 清理）最终汇入这里：

java frameworks/base/services/core/java/com/android/server/wm/WindowState.java:2501-2652（removeIfPossible，骨架+讲解注释）

```
private void removeIfPossible(boolean keepVisibleDeadWindow) {
mWindowRemovalAllowed = true;
...
try {
disposeInputChannel();
mOnBackInvokedCallbackInfo = null;
// 讲解注释①：先撤输入通道：此后 InputDispatcher
// 不会再把事件投向本窗口（§5.2 的回执链
// 断在源头，AnrTracker 计时一并解除）
boolean wasVisible = false;
if (mHasSurface && mToken.okToAnimate()) {
if (mWillReplaceWindow) {
mAnimatingExit = true;
mReplacingRemoveRequested = true;
return;
}
// 讲解注释②：分支一（替换保活）：无缝替换
//  （如界面重构）要等新窗口就位再拆旧的
wasVisible = isWinVisibleLw();
if (keepVisibleDeadWindow) {
mAppDied = true;
setDisplayLayoutNeeded();
mWmService.mWindowPlacerLocked.performSurfacePlacement();
openInputChannel(null);
getDisplayContent().getInputMonitor().updateInputWindowsLw(true);
return;
}
// 讲解注释③：分支二（死亡保活）：可见窗口的
```

```
// App 死了也不拆——保留最后画面并换新输入
// 通道（openInputChannel :2585 后立即重报
// InputDispatcher），点击它可触发应用重启
if (wasVisible) {
final int transit = (!startingWindow) ? TRANSIT_EXIT : TRANSIT_PREVIEW_DONE;
if (allowExitAnimation && mWinAnimator.applyAnimationLocked(transit, false)) {
mAnimatingExit = true;
setDisplayLayoutNeeded();
mWmService.requestTraversal();
}
...
}
// 讲解注释④：分支三（退场动画）：可见窗口先播
// TRANSIT_EXIT，真正移除推迟到动画结束后
// 的 onExitAnimationDone 再进来一次
if (mWinAnimator.getShown() && !lastWindowIsStartingWindow && isAnimating) {
mAnimatingExit = true;
setupWindowForRemoveOnExit();
if (mActivityRecord != null) {
mActivityRecord.updateReportedVisibilityLocked();
}
return;
}
}
removeImmediately();
// 讲解注释⑤：分支四（立即移除）：不可见/
// 无 Surface/无动画 → 直接清算（见下）
if (wasVisible) {
final DisplayContent displayContent = getDisplayContent();
if (displayContent.updateOrientation()) {
displayContent.sendNewConfiguration();
}
}
// 讲解注释⑥：可见窗口移除可能改变方向诉求
// （如横屏专属界面退出）→ 复查转屏（第十四章）
mWmService.updateFocusedWindowLocked(isFocused()
? UPDATE_FOCUS_REMOVING_FOCUS : UPDATE_FOCUS_NORMAL,
true /*updateInputWindows*/);
// 讲解注释⑦：焦点善后：若移除的正是焦点窗口，
// 立即重裁决并把新焦点上报（第十三章）
} finally {
Binder.restoreCallingIdentity(origId);
}
}
```

清算本身是一个"七道闸"清单：

java
frameworks/base/services/core/java/com/android/server/wm/WindowState.java:2437-2492（removeImmediately 清理清单，节选+讲解注释）

```
void removeImmediately() {
if (mRemoved) {
return;
} // 讲解注释①：幂等闸：多路径并发移除只清算一次
mRemoved = true;
```

```
// Destroy surface before super call. The general pattern is that the children
// need to be removed before the parent ...
mWinAnimator.destroySurfaceLocked(getSyncTransaction());
// 讲解注释②：先销 Surface 再拆树（子先于父）——
// 防 GraphicBuffer 泄漏的第一道闸
super.removeImmediately();
...
final DisplayContent dc = getDisplayContent();
if (isImeLayeringTarget()) {
dc.removeImeScreenshotIfPossible();
dc.setImeLayeringTarget(null);
dc.computeImeTarget(true /* updateImeTarget */);
} // 讲解注释③：若它是 IME 层级目标：撤截图、
// 清目标、重算 IME 目标（§8.4 协议线）——
// 否则键盘可能永远弹不出
if (dc.getImeInputTarget() == this && ...) {
dc.updateImeInputAndControlTarget(null);
}
...
dc.mTapExcludeProvidingWindows.remove(this);
dc.getDisplayPolicy().removeWindowLw(this);
// 讲解注释④：撤销 touch-exclude 区与策略侧登记
//  （状态栏避让等 DisplayPolicy 状态）
disposeInputChannel();
// 讲解注释⑤：输入通道二次善后（分支②③提前
// 走过也幂等）——防 fd 泄漏第二道闸：
// socketpair 两端都占 fd，InputDispatcher
// 侧还持一份 epoll 注册
mSession.windowRemovedLocked();
try {
mClient.asBinder().unlinkToDeath(mDeathRecipient, 0);
} catch (RuntimeException e) {
...
}
// 讲解注释⑥：讣告解绑：防 binderDied 对已清算
// 窗口二次触发
mWmService.postWindowRemoveCleanupLocked(this);
} // 讲解注释⑦：收尾挂账：token 子表、动画、
// 窗口列表的统一清扫入口
```

设计洞察 · 设计洞察：清理清单为什么这么长

每一行对应一类历史泄漏：②防 Surface 泄漏（GC 不回收 native 层的 GraphicBuffer）、⑤防 fd 泄漏、③防 IME 悬挂在已死目标上、⑥防"死后复活"。**系统服务的清理代码长度与它的存活年限成正比**——读 removeImmediately 比读构造函数更能理解 WMS 踩过的坑。

12.4 讣告：binderDied 与最后保活

java frameworks/base/services/core/java/com/android/server/wm/WindowState.java:3120-3140（DeathRecipient，真实代码+讲解注释）

```
private class DeathRecipient implements IBinder.DeathRecipient {
@Override
public void binderDied() {
```

```
try {
synchronized (mWmService.mGlobalLock) {
final WindowState win = mWmService
.windowForClientLocked(mSession, mClient, false);
Slog.i(TAG, "WIN DEATH: " + win);
// 讲解注释①：logcat 里这句就是"App 死了，
// WMS 开始收尸"的权威记录
if (win != null) {
if (win.mActivityRecord != null
&& win.mActivityRecord.findMainWindow() == win) {
mWmService.mTaskSnapshotController.onAppDied(win.mActivityRecord);
}
// 讲解注释②：主窗口死亡先抓任务快照——
// 最近任务里还能显示死应用最后的样子
win.removeIfPossible(shouldKeepVisibleDeadAppWindow());
// 讲解注释③：是否保活由策略决定（§12.3 分支②）：
// 可见即保活等点击复活，不可见即清算
} else if (mHasSurface) {
Slog.e(TAG, "!!! LEAK !!! Window removed but surface still valid.");
WindowState.this.removeIfPossible();
} // 讲解注释④：窗口记录已不在但 Surface 还在——
// 这行 LEAK 日志是排查 native 泄漏的金线索
}
} catch (IllegalArgumentException ex) {
// This will happen if the window has already been removed.
} // 讲解注释⑤：重复收尸是合法时序，异常静默
}
}
```

设计洞察 · 逐段精读：keepVisibleDeadWindow 的产品语义

为什么死窗口要保活？想象一个全屏游戏进程被杀：若立即清算，用户看到的是突兀的桌面；保留最后帧并换输入通道（:2585 openInputChannel(null) 新建 socketpair 后 :2586 立即重报 InputDispatcher），用户点一下屏幕，事件沿新通道上抛 system_server，触发"应用已停止/重新启动"。**一个 return 分支同时成全了视觉连续性与可恢复性**。交叉：输入通道 socketpair 原理见 §5.4 第⑤段；焦点善后细节见第十三章。

源码锚点 · 第十二章锚点

WindowState.java:1120 构造（:1143 InputWindowHandle / :1164 linkToDeath / :1184 层位 / :1214 mWinAnimator）/ :1249 attach
:2437 removeImmediately / :2501 removeIfPossible（:2533 disposeInputChannel / :2575 死亡保活 / :2585 换输入通道 / :2647 焦点善后）/ :2658 setupWindowForRemoveOnExit / :2766 openInputChannel
:3120 DeathRecipient（:3122 binderDied / :3127 WIN DEATH / :3131 LEAK 日志）

第十三章 输入窗口的户籍簿：InputMonitor 与焦点裁决

> **设计洞察 · 学习目标**
> 1. 画出从 WMS 到 InputDispatcher 的输入信息更新链：findFocusedWindow→updateFocusedWindowLocked→setInputFocusLw→updateInputWindowsLw→setInputWindowInfo；2. 解释焦点为何单独走 setFocusedWindow 事务而触摸目标走全量快照；3. 解释 updateInputWindowsLw 为何要 post 到 handler 去抖、什么场景必须 force 或立即执行；4. 用 dumpsys input 的 Focus 段对照 findFocusedWindow 的裁决条件定位焦点错乱

13.1 本章在链路中的位置

§5 章讲"事件怎么走"（七段），§6 章讲"焦点与触摸是两套逻辑"——但 WMS **如何把这两套信息报关给 InputDispatcher** 一直空着。补链路认知：InputDispatcher 并不读 WMS 的窗口树，它只认一份**平面的窗口清单**（InputWindowHandle 列表）加一个**焦点令牌**（焦点窗口的 InputChannel token）。负责汇编这份报关单的就是每屏一个的 `InputMonitor`。本章两个重点：焦点裁决（findFocusedWindow 的判定序）与全量快照（updateInputWindowsLw 的异步去抖）。交叉起点：§1.3 讲解注释⑥"必要时触发 InputMonitor 焦点更新"——本章就是那行注释的展开。

13.2 焦点裁决：findFocusedWindow 与 updateFocusedWindowLocked

焦点是"唯一窗口"的裁决，入口在 DisplayContent：

java frameworks/base/services/core/java/com/android/server/wm/DisplayContent.java:3606-3672（updateFocusedWindowLocked，骨架+讲解注释）

```
boolean updateFocusedWindowLocked(int mode, boolean updateInputWindows,
boolean topFocusedDisplayId...) {
WindowState newFocus = findFocusedWindowIfNeeded(topFocusedDisplayId);
// 讲解注释①：先裁决：newFocus 是谁?
if (mCurrentFocus == newFocus) {
return false;
} // 讲解注释②：幂等：焦点没变就收工
...
if (!mWmService.mPolicy.isScreenOn()) {
newFocus = null;
newFocus = findFocusedWindowIfNeeded(topFocusedDisplayId);
}
// 讲解注释③：灭屏时焦点强制置空——
// 系统级策略优先于任何窗口
ProtoLog.d(WM_DEBUG_FOCUS_LIGHT, "Changing focus from %s to %s displayId=%d ...",
mCurrentFocus, newFocus, getDisplayId());
final WindowState oldFocus = mCurrentFocus;
mCurrentFocus = newFocus;
// 讲解注释④：拍板：WMS 侧焦点落账
if (newFocus != null) {
...
if (newFocus.canReceiveKeys()) {
```

```
newFocus.mToken.paused = false;
} // 讲解注释⑤：获焦即解除暂停：防止有人
// pause 分发后忘了 resume 的历史 bug
}
getDisplayPolicy().focusChangedLw(oldFocus, newFocus);
// 讲解注释⑥：策略侧同步：状态栏模式、
// 系统栏 flag 等随焦点一起变
...
getInputMonitor().setInputFocusLw(newFocus, updateInputWindows);
// 讲解注释⑦：报关：把焦点令牌交给 InputMonitor
// （下节展开）——InputDispatcher 由此知道
// 按键事件该投给谁
...
scheduleToastWindowsTimeoutIfNeededLocked(oldFocus, newFocus);
return true;
}
```

裁决本体是一段"自顶向下首个可收键窗口"的遍历，判定条件浓缩成一个大 lambda：

java frameworks/base/services/core/java/com/android/server/wm/DisplayContent.java:737-800（mFindFocusedWindow，节选+讲解注释）

```
private final ToBooleanFunction<WindowState> mFindFocusedWindow = w -> {
if (!w.canReceiveKeys()) {
return false;
}
// 讲解注释①：第一关：NOT_FOCUSABLE flag、
// 死亡状态、暂停态都过不去
if (w.mIsImWindow && w.isChildWindow() && (mImeLayeringTarget == null
|| !mImeLayeringTarget.getRequestedVisibility(ITYPE_IME))) {
return false;
}
// 讲解注释②：IME 子窗特判：输入法被请求隐藏时，
// 其子窗/选择框不接管焦点
final ActivityRecord activity = w.mActivityRecord;
if (focusedApp == null) {
mTmpWindow = w;
return true;
}
// 讲解注释③：无"焦点应用"（桌面/启动中）：
// 自顶向下第一个能收键的窗口即焦点
if (!focusedApp.windowsAreFocusable()) {
mTmpWindow = w;
return true;
}
// 讲解注释④：焦点应用自身不可收键
// （如被 ANR 对话框压制）→ 同上放行
if (activity != null && w.mAttrs.type != TYPE_APPLICATION_STARTING) {
if (focusedApp.compareTo(activity) > 0) {
mTmpWindow = null;
return true;
}
// 讲解注释⑤：关键比较：本窗口所属 Activity 的
// root task 若排在焦点应用之下——"No focus
// for you!!!"（源码注释原话）：焦点不能
// 落到可见区之外的后台任务；中间还有
```

```
// TaskFragment 嵌入（双栏）特判（:794-806）
}
...
mTmpWindow = w;
return true;
}; // 讲解注释⑥：走完所有关卡未被否决的窗口
// （焦点应用自己的窗口或其上层）在此收编——
// 注意 mTmpWindow 只是"临时赢家"，返回 true
// 即停止遍历（自顶向下首个合格者）
```

> **设计洞察 · 逐段精读：焦点裁决是"层级判断"而非"坐标判断"**
>
> 六个关卡全部基于窗口树与 token 关系，与手指位置无关——这正是 §6"触摸是命中、按键是焦点"的服务端实现。调试焦点错乱时按关卡逐条排除：dumpsys window 里先看 mCurrentFocus 与 mFocusedApp 是否一致（关卡⑤⑥），再看窗口 flags（关卡①），最后看 IME 可见性请求（关卡②）。dumpsys input 的 Focus: 段显示的则是报关结果——若它与 mCurrentFocus 不一致，问题在 §13.3 的更新链上。

13.3 全量快照：updateInputWindowsLw 与报关事务

焦点之外，InputDispatcher 还需要每个窗口的**触摸属性**（可见区、可触碰区、遮挡模式、是否 spy）——这份清单每次布局/可见性变化都要重报：

java frameworks/base/services/core/java/com/android/server/wm/InputMonitor.java:324-343（更新入口与去抖，真实代码+讲解注释）

```
void updateInputWindowsLw(boolean force) {
if (!force && !mUpdateInputWindowsNeeded) {
return;
}
// 讲解注释①：双重闸：非 force 且无人申请过
// 更新 → 直接返回（省一次 post）
scheduleUpdateInputWindows();
}
private void scheduleUpdateInputWindows() {
if (mDisplayRemoved) {
return;
}
if (!mUpdateInputWindowsPending) {
mUpdateInputWindowsPending = true;
mHandler.post(mUpdateInputWindows);
}
}
// 讲解注释②：去抖：同一轮布局里 N 个窗口变化
// 只 post 一次，稍后统一汇编全量快照——
// 全量快照本身是 O(窗口数)，必须合并
```

真正干活的是这个 Runnable 与它的消费者：

java frameworks/base/services/core/java/com/android/server/wm/InputMonitor.java:105-129（UpdateInputWindows，真实代码+讲解注释）

```
private class UpdateInputWindows implements Runnable {
@Override
public void run() {
```

```
synchronized (mService.mGlobalLock) {
mUpdateInputWindowsPending = false;
mUpdateInputWindowsNeeded = false;
if (mDisplayRemoved) {
return;
}
// Populate the input window list with information about all of the windows
// that could potentially receive input. ...
// If there's a drag in flight, provide a pseudo-window to catch drag input
final boolean inDrag = mService.mDragDropController.dragDropActiveLocked();
// 讲解注释①：拖拽是"伪窗口"：它不是窗口树
// 成员，但需要参与触摸命中——全量快照
// 特意给它留了席位
mUpdateInputForAllWindowsConsumer.updateInputWindows(inDrag);
}
}
}
// 讲解注释②：随后 forAllWindows 自顶向下遍历，
// 对每个窗口调 populateInputWindowHandle（:241）
// 填充区域/标志，再 setInputWindowInfoIfNeeded
//  （:644）挂进 mInputTransaction
```

焦点令牌的报关在同一个 Runnable 的收尾处：

java frameworks/base/services/core/java/com/android/server/wm/InputMonitor.java:398-466（updateInputFocusRequest 与 requestFocus，骨架+讲解注释）

```
private void updateInputFocusRequest(InputConsumerImpl recentsAnimationInputConsumer) {
final WindowState focus = mDisplayContent.mCurrentFocus;
// 讲解注释①：报关的"货物"是 §13.2 裁决好的
// mCurrentFocus——注意本章不重新裁决
if (recentsAnimationInputConsumer != null && focus != null) {
...
if (shouldApplyRecentsInputConsumer) {
requestFocus(recentsAnimationInputConsumer.mWindowHandle.token,
recentsAnimationInputConsumer.mName);
return;
}
}
// 讲解注释②：最近任务动画期间：焦点改道给
// recents 输入消费者——事件收不到的另一个
// 高频原因（动画进行中）
final IBinder focusToken = focus != null ? focus.mInputChannelToken : null;
if (focusToken == null) {
...
if (mDisplayContent.mFocusedApp != null && mInputFocus != null) {
mInputTransaction.removeCurrentInputFocus(mDisplayId);
}
mInputFocus = null;
return;
}
// 讲解注释③：应用有焦点但窗口未就绪：
// 主动撤销 InputDispatcher 侧焦点——
// 保证两边账本严格一致
if (!focus.mWinAnimator.hasSurface() || !focus.mInputWindowHandle.isFocusable()) {
mInputFocus = null;
```

```
return;
}
// 讲解注释④：没有 Surface 的焦点窗口同样
// 不能收键——窗口存在与可收输入解耦时
// 以"可收"为准
requestFocus(focusToken, focus.getName());
}
private void requestFocus(IBinder focusToken, String windowName) {
if (focusToken == mInputFocus) {
return;
}
mInputFocus = focusToken;
mInputTransaction.setFocusedWindow(mInputFocus, windowName, mDisplayId);
// 讲解注释⑤：焦点单独走 setFocusedWindow：
// 一条 SF Transaction 与窗口层位变化
// 原子提交，InputDispatcher 在事务应用时
// 收到焦点与窗口清单的一致快照
...
}
```

设计洞察 · 设计洞察：为什么焦点与触摸走两条通道

触摸目标是"多对多"（split touch、spy 窗口、遮挡模式），只能随全量清单重报；焦点是"一对一"且对延迟敏感（按键 immediacy），单独 setFocusedWindow 让它不必等整份清单重建。而全量快照的 post 去抖与 force 直通（updateInputWindowsImmediately :346）并存，是因为有极少数路径（如死亡保活 : 2586）等不及下一轮 Handler 调度。**读代码时看到 force/Immediately 形参，先问"哪个时序等不起"**——答案就是设计动机。

源码锚点 · 第十三章锚点

DisplayContent.java:737 mFindFocusedWindow / :3576 findFocusedWindowIfNeeded / :3606 updateFocusedWindowLocked（:3634 落账 / :3663 setInputFocusLw）
InputMonitor.java:105 UpdateInputWindows / :241 populateInputWindowHandle / :324 updateInputWindowsLw / :331 scheduleUpdateInputWindows / :358 setInputFocusLw / :398 updateInputFocusRequest / :458 requestFocus（:464 setFocusedWindow）/ :644 setInputWindowInfoIfNeeded
RootWindowContainer.java:470 updateFocusedWindowLocked（多屏循环入口）

第十四章 转屏全链路：冻结、旋转与解冻

设计洞察 · 学习目标

1. 走通 rotationForOrientation→updateRotationUnchecked→prepareNormalRotationAnimation→sendNewConfiguration→applyRotation 的完整链；2. 说出普通转屏（截图冻结）与 seamless 转屏的分界条件；3. 解释转屏期间输入为何被冻结、窗口为何分批重绘、解冻要等哪些条件；4. 用 Screen frozen for 与 WINDOW_FREEZE_TIMEOUT 日志定位转屏黑屏与长冻屏

14.1 本章在链路中的位置

转屏是本册前半部所有机制的**压力测试**：每个窗口都要走一次 relayout（§2.2），主线程要重跑三幕剧（§3.3），输入要在"旧方向坐标"与"新方向坐标"之间切换（§5.1 的坐标变换），IME 截图与焦点都要重新裁决（第十三章）。WMS 的应对是一个三段式仪式：**决策（选哪个方向）→ 冻结（截屏盖住屏幕、冻结输入分发）→ 旋转与解冻（逐窗重绘、撤截图）**。载体是 DisplayContent 的搭档 `DisplayRotation`——每个 DisplayContent 构造时都配一个（DisplayContent.java:1074）。

14.2 决策层：DisplayRotation.updateRotationUnchecked

java frameworks/base/services/core/java/com/android/server/wm/DisplayRotation.java:420-546（updateRotationUnchecked，骨架+讲解注释）

```
boolean updateRotationUnchecked(boolean forceUpdate) {
final int displayId = mDisplayContent.getDisplayId();
if (!forceUpdate) {
if (mDeferredRotationPauseCount > 0) {
// Rotation updates have been paused temporarily. ...
return false;
}
// 讲解注释①：暂停门：窗口拖拽、伪旋转等场景
// 主动挂起转屏（pause/resume 成对使用）
final ScreenRotationAnimation screenRotationAnimation =
mDisplayContent.getRotationAnimation();
if (screenRotationAnimation != null && screenRotationAnimation.isAnimating()) {
return false;
}
// 讲解注释②：动画门：上一次转屏动画没播完，
// 本轮更新推迟到解冻后再试
if (mService.mDisplayFrozen) {
return false;
}
// 讲解注释③：冻结门：屏还冻着（动画播完但
// 尚未解冻）也推迟——三门保证转屏串行化
if (mDisplayContent.mFixedRotationTransitionListener.shouldDeferRotation()) {
...
return false;
}
// 讲解注释④：recents 门：最近任务动画里
// 关闭中的应用不应再通过传感器触发旋转
}
if (!mService.mDisplayEnabled) {
return false;
} // 讲解注释⑤：屏幕未使能，选了也没意义
```

```
final int oldRotation = mRotation;
final int lastOrientation = mLastOrientation;
final int rotation = rotationForOrientation(lastOrientation, oldRotation);
// 讲解注释⑥：核心决策：综合显示器默认方向、
// 焦点应用 requestedOrientation、传感器
// （免费旋转开关）算出目标 rotation（:1128）
if (oldRotation == rotation) {
// No change.
return false;
}
...
if (recentsAnimationController != null) {
recentsAnimationController.cancelAnimationForDisplayChange();
}
// 讲解注释⑦：转屏与 recents 动画互斥：
// SysUI 承受不了中途换向——先取消再说
if (deltaRotation(oldRotation, rotation) != Surface.ROTATION_180) {
mDisplayContent.mWaitingForConfig = true;
}
// 讲解注释⑧：180 度转屏宽高不变、跳过等配置；
// 其余转角必须等"新配置落地"才允许解冻
mRotation = rotation;
mDisplayContent.setLayoutNeeded();
if (mDisplayContent.mTransitionController.isShellTransitionsEnabled()) {
...
return true;
}
// 讲解注释⑨：Shell 过渡模式（13 默认）：转屏
// 作为一个 Transition 走 shell 编排，本函数
// 到此收工
mService.mWindowsFreezingScreen = WINDOWS_FREEZING_SCREENS_ACTIVE;
mService.mH.sendNewMessageDelayed(WindowManagerService.H.WINDOW_FREEZE_TIMEOUT,
mDisplayContent, WINDOW_FREEZE_TIMEOUT_DURATION);
// 讲解注释⑩：旧路径：登记"窗口冻结中"并上
// 超时闹钟——闹钟是防止个别窗口永远不重绘
// 导致屏幕永久冻结的保险丝
if (shouldRotateSeamlessly(oldRotation, rotation, forceUpdate)) {
prepareSeamlessRotation();
} else {
prepareNormalRotationAnimation(); // → :615 起调用 WMS.startFreezingDisplay
}
// 讲解注释⑪：分岔：seamless（无冻结、各窗口
// 独立旋转 Surface，如相机/折叠屏）或普通
// （截屏冻结整屏）——14.3 展开
startRemoteRotation(oldRotation, mRotation);
return true;
}
```

设计洞察 · 设计洞察：三门（暂停/动画/冻结）本质是"转屏串行化"

转屏是全屏事务，并发转屏必然产生方向撕裂。三个门各挡一类重入：人为暂停、动画未完、冻结未解。加上 mWindowsFreezingScreen 与 WINDOW_FREEZE_TIMEOUT 闹钟，构成"串行 + 保险丝"的完整模式。**凡涉及全局状态的批量变更，都值得学这套防御**：宁可推迟，不可交叠。

14.3 冻结账本：WMS 的 startFreezingDisplay 与 stopFreezingDisplayLocked

普通转屏在 prepareNormalRotationAnimation 里调用 WMS 记账：

java frameworks/base/services/core/java/com/android/server/wm/WindowManagerService.java:6041-6079（doStartFreezingDisplay，节选+讲解注释）

```
private void doStartFreezingDisplay(int exitAnim, int enterAnim, DisplayContent
displayContent,
int overrideOriginalRotation) {
ProtoLog.d(WM_DEBUG_ORIENTATION,
"startFreezingDisplayLocked: exitAnim=%d enterAnim=%d called by %s",
exitAnim, enterAnim, Debug.getCallers(8));
mScreenFrozenLock.acquire();
mAtmService.startLaunchPowerMode(POWER_MODE_REASON_CHANGE_DISPLAY);
// 讲解注释①：进入转屏电源模式：告诉功耗框架
// "接下来的 CPU 高占用是转屏的"
mDisplayFrozen = true;
mDisplayFreezeTime = SystemClock.elapsedRealtime();
...
mFrozenDisplayId = displayContent.getDisplayId();
// 讲解注释②：冻结开始：记时间戳与冻结屏 id，
// mDisplayFrozen 防多屏同时冻结（:692 定义）
mInputManagerCallback.freezeInputDispatchingLw();
// 讲解注释③：输入冻结：InputDispatcher 暂停
// 本屏分发（:6128 解冻时恢复）——转屏期间
// 手指坐标参考系正在换，宁可丢弃也不误投
if (displayContent.mAppTransition.isTransitionSet()) {
displayContent.mAppTransition.freeze();
}
...
displayContent.updateDisplayInfo();
final int originalRotation = ...;
displayContent.setRotationAnimation(new ScreenRotationAnimation(displayContent,
originalRotation));
} // 讲解注释④：截屏：ScreenRotationAnimation 抓
// 当前屏幕内容做成 Surface 盖在最上层——
// 用户看到的"转屏动画"其实是这张截图在转
```

解冻不是定时器，而是"条件收齐"：

java frameworks/base/services/core/java/com/android/server/wm/WindowManagerService.java:6082-6130（stopFreezingDisplayLocked，骨架+讲解注释）

```
void stopFreezingDisplayLocked() {
if (!mDisplayFrozen) {
return;
}
final DisplayContent displayContent = mRoot.getDisplayContent(mFrozenDisplayId);
final int numOpeningApps;
final boolean waitingForConfig;
final boolean waitingForRemoteRotation;
...
if (waitingForConfig || waitingForRemoteRotation || mAppsFreezingScreen > 0
|| mWindowsFreezingScreen == WINDOWS_FREEZING_SCREENS_ACTIVE
```

```
|| mClientFreezingScreen || numOpeningApps > 0) {
ProtoLog.d(WM_DEBUG_ORIENTATION, "stopFreezingDisplayLocked: Returning "
+ "waitingForConfig=%b, waitingForRemoteRotation=%b, ...");
return;
}
// 讲解注释①：五条件收齐才解冻：新配置已落地、
// 远程旋转决策已回、应用/窗口/客户端三方
// 冻结全部解除、没有开屏中的应用——
// 任何一条不满足都继续冻着
doStopFreezingDisplayLocked(displayContent);
...
}
private void doStopFreezingDisplayLocked(DisplayContent displayContent) {
...
mFrozenDisplayId = INVALID_DISPLAY;
mDisplayFrozen = false;
mInputManagerCallback.thawInputDispatchingLw();
// 讲解注释②：解冻输入：积压的输入事件按新
// 方向坐标重新分发
mLastDisplayFreezeDuration = (int)(SystemClock.elapsedRealtime() - mDisplayFreezeTime);
StringBuilder sb = new StringBuilder(128);
sb.append("Screen frozen for ");
TimeUtils.formatDuration(mLastDisplayFreezeDuration, sb);
if (mLastFinishedFreezeSource != null) {
sb.append(" due to ");
sb.append(mLastFinishedFreezeSource);
}
ProtoLog.i(WM_ERROR, "%s", sb.toString());
// 讲解注释③：logcat 的"Screen frozen for 380ms
// due to ..."——转屏耗时排查的第一手数据，
// source 字段说明是谁拖住了最后一次解冻
mH.removeMessages(H.APP_FREEZE_TIMEOUT);
mH.removeMessages(H.CLIENT_FREEZE_TIMEOUT);
...
} // 讲解注释④：清保险丝闹钟：APP/CLIENT_FREEZE_
// TIMEOUT 与 WINDOW_FREEZE_TIMEOUT 三兄弟
```

新配置落地后由 DisplayContent 完成真正的旋转：

java frameworks/base/services/core/java/com/android/server/wm/DisplayContent.java:1980-2034（applyRotation，骨架+讲解注释）

```
private void applyRotation(final int oldRotation, final int rotation) {
mDisplayRotation.applyCurrentRotation(rotation);
// 讲解注释①：落账：rotation 进历史记录
//  （applyCurrentRotation :356），此后
// getRotation() 返回新值
...
updateDisplayAndOrientation(getConfiguration().uiMode, null /* outConfig */);
// 讲解注释②：交换宽高：displayInfo 从此是新
// 方向——App 端 relayout 拿到的就是新尺寸
if (screenRotationAnimation != null && screenRotationAnimation.hasScreenshot()) {
screenRotationAnimation.setRotation(transaction, rotation);
}
// 讲解注释③：让盖在最上面的截图开始转——
// 用户此刻感知到的"屏幕在转"
```

```
if (!shellTransitions) {
forAllWindows(w -> {
w.seamlesslyRotateIfAllowed(transaction, oldRotation, rotation, rotateSeamlessly);
}, true /* traverseTopToBottom */);
...
}
// 讲解注释④：seamless 窗口的 Surface 立即按新
// 方向变换（缓冲内容跟着转）；普通窗口
// 不在此列——它们等 App 重绘
mWmService.mDisplayManagerInternal.performTraversal(transaction);
scheduleAnimation();
forAllWindows(w -> {
if (!w.mHasSurface) return;
if (!rotateSeamlessly) {
w.setOrientationChanging(true);
}
w.mReportOrientationChanged = true;
}, true /* traverseTopToBottom */);
// 讲解注释⑤：给普通窗口挂"方向已变"标记：
// 下一次 performTraversals（§3.3）会带着新
// 尺寸走 relayout，重绘完成后逐窗报告，
// 全部报告完才允许 §14.3 的解冻五条件收齐
for (int i = mWmService.mRotationWatchers.size() - 1; i >= 0; i--) {
...
rotationWatcher.mWatcher.onRotationChanged(rotation);
...
}
} // 讲解注释⑥：广播转屏：壁纸、SystemUI 等
// rotationWatcher 在此收到通知
```

设计洞察 · 逐段精读：转屏黑屏的两种病与一个日志

黑屏一：截图冻住了但窗口迟迟不重绘——解冻五条件里的 mWindowsFreezingScreen 迟迟不清零，WINDOW_FREEZE_TIMEOUT（约 2 秒，WINDOW_FREEZE_TIMEOUT_DURATION）强制解冻；黑屏二：App 收到新尺寸但自身渲染慢，解冻后画面仍是旧方向。排查入口统一是 logcat：`grep -E "Screen frozen for|FREEZE_TIMEOUT|startFreezingDisplay"` ——冻结时长与"due to"来源直接指出责任方（配置未回/应用未画完/远程旋转未决）。再配合 `dumpsys window displays` 看 mDisplayFrozen 与 rotation 现值。交叉：冻结期间输入暂停与 §5 的 AnrTracker 不冲突——暂停分发即暂停计时。

源码锚点 · 第十四章锚点

DisplayRotation.java:378 updateOrientation / :420 updateRotationUnchecked（:439 冻结门 / :468 rotationForOrientation（定义 :1128）/ :615 prepareNormalRotationAnimation）/ :649 isRotatingSeamlessly / :881 freezeRotation / :886 thawRotation
WindowManagerService.java:6013 startFreezingDisplay / :6041 doStartFreezingDisplay（:6051 冻结 / :6059 输入冻结 / :6078 截屏）/ :6082 stopFreezingDisplayLocked（五条件）/ :6120 doStop（:6127 解冻 / :6128 输入解冻）
DisplayContent.java:1434 sendNewConfiguration（mWaitingForConfig 清零）/ :1980 applyRotation（:2015 setOrientationChanging）

教科书习题集（分级）

第一级：简答题（检验概念理解）

1. **同步屏障的物理形态是什么？由谁插入、谁移除?**

参考答案要点：一条 target 为 null 的 Message（token 存于 arg1）；scheduleTraversals（ViewRootImpl.java:2260）插入，doTraversal（:2280）或 unscheduleTraversals（:2271）移除；它只拦同步消息，异步消息照常通行。

2. **Choreographer 四车道固定顺序是什么？为什么不能乱?**

参考答案要点：INPUT→ANIMATION→INSETS→TRAVERSAL（Choreographer.java:226-270）；同帧"先消化输入再画画面"形成因果闭环，乱序会让触控响应恒定晚一帧。

3. **焦点与触摸目标是两套什么逻辑?**

参考答案要点：焦点=按键寻址，由 WMS 依层级裁决（每屏唯一，findFocusedWindow）；触摸=可见区命中测试（InputDispatcher 侧，可多目标 split）。前者是"层级判断"，后者是"坐标判断"。

4. **输入事件为什么走 socketpair 而不是 Binder?**

参考答案要点：高频小包（200Hz 触摸流）下 Binder 每次事务的组包开销不划算；分发线程不能被 Binder 线程池策略绑架；socket+epoll 是实时 IPC 的成熟解。

5. **addWindow 四步审验漏斗是什么？对应哪两类开发者崩溃?**

参考答案要点：屏→token→权限→建籍；token 不匹配对应 BadTokenException（典型：对话框早于 Activity attach），系统类型缺权限对应 ADD_PERMISSION_DENIED。

6. **removeImmediately 清理清单中任意三道闸各防什么泄漏?**

参考答案要点：destroySurfaceLocked 防 GraphicBuffer 泄漏；disposeInputChannel 防 fd 泄漏；IME target 清理防键盘永远弹不出；unlinkToDeath 防"死后复活"。（任答三条即可）

7. **三缓冲比双缓冲多买了什么?**

参考答案要点：一份缓冲内存换来"容忍一次抖动"：App 慢一帧时 SF 仍有缓冲可合成，避免双缓冲互等导致的帧率腰斩。

8. **InputStage 责任链顺序？IME 为什么排在 View 之前?**

参考答案要点：Native→ViewPreIme→Ime→ViewPostIme→Synthetic→View；输入法截胡是设计而非 bug：文本输入场景 IME 必须先解释按键，App 只是消费者之一。

9. **mFirstTouchTarget 的四条规则?**

参考答案要点：拦截权在父（onInterceptTouchEvent）；命中权在子（hit test）；锁定起于 DOWN（后续 MOVE/UP 全归目标）；消费可中途易主（每轮 MOVE 复议拦截）。

10. **转屏为什么必须冻结输入分发？**

参考答案要点：转屏期间坐标参考系在换（InputReader 变换矩阵、窗口可见区全部重算），继续分发会误投或坐标错乱；freezeInputDispatchingLw 到 thawInputDispatchingLw 之间宁可丢弃也不误投。

第二级：分析题（检验机制理解）

11. **追踪调用链**：从 VSync 到达 FrameDisplayEventReceiver.onVsync 起，列出消息抵达 doFrame 的完整路径（含屏障穿越）。

参考答案要点：onVsync（Choreographer.java:1172）→ msg.setAsynchronous(true)（:1204）→ sendMessageAtTime 按 vsync 时刻入队 → MessageQueue.next()（:319）遇 target==null 屏障（:341）跳过同步消息取走异步消息 → run()（:1210）→ doFrame 四车道。

12. **失败推演**：某次 traversal 异常导致 removeSyncBarrier 永远没被调用，系统表现是什么？

参考答案要点：屏障悬空→next() 只能取异步消息；主线程同步消息全部饿死→假死（进程存活、Binder 健康）→输入回执超时演变为 ANR。诊断靠 debuggerd 主线程栈（停在 nativePollOnce）与 Skipped frames 日志。

13. **对比分析**：addWindow 的四步审验与 InputDispatcher 的事件寻址如何衔接？

参考答案要点：addWindow 第四步建籍（WindowState 构造）装配 InputWindowHandle 并 openInputChannel；InputMonitor.updateInputWindowsLw 把句柄清单与焦点令牌报关；Dispatcher 只认报关单、不读窗口树——审验决定"能不能进户口"，报关决定"事件往哪投"。

14. **对比分析**：焦点与触摸目标在 InputMonitor 的报关方式差异及原因。

参考答案要点：焦点一对一且延迟敏感→单独 setFocusedWindow 事务（InputMonitor.java:464）；触摸多对多（split/spy/遮挡模式）→全量快照 setInputWindowInfo；全量快照用 handler post 去抖，极端时序用 force/updateInputWindowsImmediately 直通。

15. **追踪调用链**：InsetsController.show(ITYPE_IME) 从 App 到 WMS 再回到 App 的往返。

参考答案要点：show（InsetsController.java:926）→ applyAnimation（:1435）→ controlAnimationUnchecked（:1043）collectSourceControls 等控制权；App 侧 updateRequestedVisibilities（:1409）→ Session.updateRequestedVisibilities（:697）→ WMS 侧 InsetsPolicy/InsetsStateController 重算控制权 → onControlsChanged（:847）回发 → InsetsAnimationControlImpl 播动画（InsetsAnimationThread 不受主线程卡顿影响）。

16. **失败推演**：转屏时某 App 主线程死循环拒绝重绘，系统有哪几道保险丝？

参考答案要点：WINDOW_FREEZE_TIMEOUT（约 2s）强制解冻窗口冻结；APP/CLIENT_FREEZE_TIMEOUT 处理应用级冻结；解冻日志 mLastFinishedFreezeSource 记录责任来源；输入解冻后该 App 事件堆积，5s 后输入 ANR 收尾。

17. **对比分析**：WindowToken 与 InputChannelToken 两个 token 各自解决什么问题?

参考答案要点：WindowToken 是户口（准入+层位分组，addWindow 审验对象）；InputChannelToken 是邮寄地址（InputDispatcher 寻址焦点与目标窗口，requestFocus（InputMonitor.java:458）与 setFocusedWindow 用的都是它）。

第三级：设计题

18. **架构演进**：若重设计输入通道（socketpair 方案），你会保留什么、改进什么?

参考答案要点：保留：fd 传递、epoll 语义、回执协议（ANR 计时依赖它）；改进方向：移动事件批量合并、回执捎带在下一个事件包里（减少 syscall）、通道生命周期与窗口销毁更强绑定。言之成理即可。

19. **架构演进**：为折叠屏设计"转形态不黑屏"策略，列出约束与机制。

参考答案要点：优先 seamless 路径（shouldRotateSeamlessly：Surface 无缝变换+窗口独立缓冲），约束：宽高变化必须 relayout、相机等敏感窗口免冻结、shell transition 收编转屏动画；兜底：截图冻结+超时闹钟强制解冻。

20. **排障方案**：用户反馈"点击偶发无响应"，设计分层定位方案。

参考答案要点：第一层 getevent 确认硬件事件在；第二层 dumpsys input/dumpsys window 看焦点、通道、ANR 计时与投递对象；第三层 debuggerd/perfetto 看 App 主线程（InputStage 卡点还是业务卡）。结论四选一：没事件/投错人/投了没回执/回执慢。

术语表（按拼音排序）

术语	解释
BufferQueue	三缓冲轮转的生产者-消费者队列：App 产缓冲、SF 消缓冲，槽位状态机决定每帧谁持有画布
Choreographer	编舞者：App 侧帧调度器，维护 INPUT/ANIMATION/INSETS/TRAVERSAL/COMMIT 五条回调车道，VSync 到达时依序发车
DisplayContent	WMS 侧一块屏的容器：窗口树、DisplayRotation、InputMonitor 各配一份，多屏即多实例
DisplayList	硬件加速的绘制命令录制结果：UI 线程录、RenderThread 放，局部失效只重录子树

术语	解释
FrameDisplayEventReceiver	Choreographer 的 VSync 接收端：onVsync 把硬件时刻重投为异步消息，"异步"通行证的发放现场
InputChannel	socketpair 封装：server 端归 InputDispatcher、client 端随 addWindow 交给 App，输入专用实时通道
InputMonitor	每屏一个的输入报关员：汇编 InputWindowHandle 全量清单与焦点令牌，经 SF 事务上报 InputDispatcher
InputWindowHandle	WMS 向 InputDispatcher 描述单窗口输入属性的句柄：可见区、可触碰区、焦点能力、遮挡模式
InsetsSource	单一遮挡源（状态栏/导航栏/IME 等，按 type 分）的状态记录：frame、可见性、控制权目标
InsetsState	一屏全部 InsetsSource 的集合，随 relayout 分发到窗口，是"内容区谈判"的底账
MessageQueue 同步屏障	target 为 null 的特殊消息：拦住同步消息、只放行异步消息；postSyncBarrier 与 removeSyncBarrier 必须成对
VSync	垂直同步节拍：App 与 SF 双相位错开（双 VSync 模型），是本册一切"帧时刻"的基准
WindowState	WMS 侧窗口事实载体：Frame/Insets/可见性/SurfaceControl/输入句柄；出生（构造）与死亡（removeIfPossible）都是重头戏
WindowStateAnimator	WindowState 的动画与 Surface 生命周期伴随者：destroySurfaceLocked 与退场动画的执行者
层位（layer）	z 序数值：type 层基线乘倍率加偏移（WindowState.java:1184），同层内按 token 与窗口顺序再排
触摸锁定	DOWN 时刻决定 mFirstTouchTarget，后续 MOVE/UP 全归它；父容器拦截是唯一合法的易主方式
点击穿透	"点了没反应"的机制学归因：hit test 选中了别的窗口（遮挡/区域外/消费者优先），而非事件丢失
掉帧分责	FrameTimeline 把 jank 标给 App/RenderThread/SF/Display 四方，配合三缓冲模型读责任链
冻结（转屏）	截屏盖屏+输入停发：转屏串行化的代价；解冻要等配置/远程旋转/三方冻结等条件收齐
焦点	按键事件寻址：WMS 裁决（findFocusedWindow）+ InputDispatcher 执行，每屏唯一且随窗口生死迁移
死亡保活	可见窗口的 App 死后保留最后画面并换输入通道（WindowState.java:2575），点击可触发应用复活
输入消费者	InputConsumerImpl：拖拽、最近任务等"非窗口触点"以伪窗口身份参与触摸命中

术语	解释
悬空屏障	只插不拔的同步屏障：next() 找不到异步消息而无限睡，主线程假死但进程存活
异步消息	打过 setAsynchronous(true) 的消息：唯一能穿越同步屏障的消息；Choreographer 自产消息全部异步

排障手册：WMS 窗口与输入体系问题定位

定位总原则：先定界（硬件→内核→native→framework→App），再归因，后修复。本册域内的证据源有四：dumpsys window（户口与层序）、dumpsys input（寻址与计时）、dumpsys SurfaceFlinger（合成与层）、logcat 的 WindowManager/InputDispatcher tag。

场景一：点击无响应

症状：按钮可见、按下无反馈，或偶发"点了没反应"。

定位命令：`adb shell getevent /dev/input/event3` 确认内核层有事件；`adb shell dumpsys input | grep -A 10 Focus` 看焦点窗口是谁；`adb shell dumpsys window windows | grep -B 2 -A 8 mCurrentFocus`；持续 5s 无响应看 logcat 的 ANR 记录。

根因：四选一——事件没到（硬件/驱动）、投错人（焦点或命中在别的窗口：透明遮挡层、spy 窗口、动画期焦点改道）、投了没回执（主线程卡）、回执慢（业务卡）。

修复：遮挡层加 FLAG_NOT_TOUCHABLE 或收窄区域；主线程卡抓 trace 定位卡点；命中异常用布局检查器对照窗口可见区与可触碰区。

场景二：焦点错乱（按键/输入法打到错误窗口）

症状：对话框弹出后按返回键作用于底层界面；输入法打字进了后台输入框。

定位命令：`adb shell dumpsys window | grep -E "mCurrentFocus|mFocusedApp"`；`adb shell dumpsys input | grep -i focus`；`adb logcat -s WindowManager` 看 Focus changing 记录。

根因：findFocusedWindow 裁决与直觉不符：NOT_FOCUSABLE flag、focusedApp 层级比较未命中、窗口无 Surface 不报关焦点（第十三章讲解注释④）、动画期焦点改道 recents 消费者。

修复：修窗口 flag 与 token 归属；对话框使用同 token 的应用窗口类型；改完后用 dumpsys 复验 mCurrentFocus 与 input Focus 一致。

场景三：转屏黑屏或长冻屏

症状：旋转后黑屏 1-3 秒以上，个别设备永久黑屏需重启。

定位命令：`adb logcat | grep -E "Screen frozen for|FREEZE_TIMEOUT|startFreezingDisplay"`；`adb shell dumpsys window displays | grep -E "mDisplayFrozen|rotation"`；`adb shell dumpsys activity top | grep orientation`。

根因：解冻五条件之一未收齐——某窗口未报告重绘（mWindowsFreezingScreen 挂起）、App 配置未回（relaunch 循环）、远程旋转未决；超 WINDOW_FREEZE_TIMEOUT 会强制解冻但画面可能仍是旧方向。

修复：确认 mOrientationChanging 挂着的责任窗口；App 侧避免在 configuration change 中同步做重活；系统侧排查两个窗口交替请求不同方向导致的震荡。

场景四：状态栏/键盘遮挡内容（Insets 协商失败）

症状：列表被状态栏压住；键盘弹出后输入框不动；或键盘收起后底部留白。

定位命令：`adb shell dumpsys window displays | grep -A 40 InsetsState` 看各 type 的 frame 与可见性；`adb logcat -s InsetsController InsetsPolicy`；App 侧在 onApplyWindowInsets 打日志。

根因：窗口没消费对应 type 的 insets（decorFitsSystemWindows/behavior 配置错）；IME target 裁决失败或目标未清（第十二章讲解注释③）；adjustResize 下 insets 未进入测量参数。

修复：明确 setOnApplyWindowInsetsListener 的消费返回值；键盘跟随用 WindowInsetsAnimationCompat；键盘不动先查 IME target 是否指向本窗口。

场景五：输入卡顿（触摸延迟/滑动掉帧）

症状：滑动跟手性差、按下高亮延迟、偶发跳变。

定位命令：**adb shell dumpsys gfxinfo 包名 framestats**；`adb shell dumpsys input` 看事件流时间戳间隔；perfetto FrameTimeline 按 §4.3 分责。

根因：App 慢（traversal 超预算）、RenderThread 慢（重录/GPU）、SF 慢（层数多无 Overlay 直通）、分发链慢（InputStage 上挂了监听器或无障碍过滤器改写事件）。

修复：按责任方修复；分发链问题排查已安装的无障碍服务与事件监听器。

场景六：键盘弹出瞬间主线程长卡

症状：点输入框后几十到几百毫秒明显掉帧，键盘动画生硬。

定位命令：`adb logcat -s InsetsController`（IC.showRequestFromApi 等 async trace 起止）；perfetto 中对齐 IME show 链路与 traversal 段，看是否同帧挤兑。

根因：IME 控制权就绪（onControlsChanged）与 insets 变化触发的 relayout 挤在同一帧；PendingControlRequest 超时后回退默认动画。

修复：用 WindowInsetsAnimationControlListener 预取控制权自行驱动动画；降低 IME target 窗口的 relayout 成本（减层次、定高行）。

→ 深入：第十一章 §11.5（悬空屏障）与第十四章 §14.3（冻结账本）是同类排查的深水区。

附录 E：窗口/输入/绘制核心机制深度速查表

一张表看完整个 WMS/输入/绘制体系

机制	一句话	源码锚点	所在册
窗口层级	RootWindowContainer→DisplayContent→WindowToken→WindowState	WMS.java:1449	本册
addWindow 审验	屏→token→权限→建籍→配对输入通道	WMS.java:1449	本册
relayout	给 SurfaceControl（合成器侧句柄）	WMS.java:2219	本册
z 序	type 层基线+同层 token 序+动画暂存	WMS 层序器	本册
Insets	InsetsController 与 WMS 按 type 协商	InsetsController.java:79	本册
performTraversals	measure→layout→draw 三幕剧	ViewRootImpl.java:2286	本册
同步屏障	scheduleTraversals 插入/doTraversal 移除	ViewRootImpl.java:2257	本册
Choreographer	INPUT→ANIMATION→INSETS→TRAVERSAL 四车道	Choreographer.java:226	本册
双 VSync	App 与 SF 相位错开	EventThread/Scheduler	本册/书8
三缓冲	写/排/读/垫 四角色的最小完备配置	BufferQueue	本册/书8
硬件加速	UI 线程录 DisplayList→RT 执行 GPU	RenderThread/libhwui	本册
触摸七段	内核→EventHub→Reader→Dispatcher→socket→ViewRootImpl→View	inputflinger	本册
触摸锁定	DOWN 决定 mFirstTouchTarget（后续 MOVE/UP 归它）	ViewGroup.java	本册
InputStage	Native→PreIme→Ime→ViewPostIme→Synthetic 责任链	ViewRootImpl	本册
ANR 5s	publishMotion 起计时→finishInputEvent 摘除	AnrTracker.cpp	本册/书4

机制	一句话	源码锚点	所在册
焦点与触摸	焦点=按键寻址（唯一），触摸=命中测试（可 split）	InputDispatcher	本册
手势导航	手势区命中→系统合成 KEYCODE_BACK	Dispatcher/Policy	本册
预测返回	手势按下即发渐进回调（progress/start/cancel）	OnBackInvokedDispatcher:28	本册
传感器 FMQ	EventQueue 共享内存环形队列+旗标位	ISensors.aidl:129	本册
IME 文本协议	InputConnection 每次=binder 往返	InputMethodManagerService	本册
SurfaceView	独立 Surface+打洞+独立节拍	SurfaceView.java:239	本册
RenderNode	属性改矩阵不重录 Display List	libhwui	本册/书8
FrameTimeline	期望呈现 vs 实际呈现+责任标签	Perfetto	本册/书8